OblivRec: Privacy-Preserving Recommendation with Private Training and Private Delivery

Mainak Sarkar (230619) Shrasti Dwivedi (230982) Shravan Agrawal (230984)

CS670: Cryptographic Techniques for Privacy Preservation
Indian Institute of Technology Kanpur
6 November 2026

Abstract

A recommender learns from what its users consume, so a private recommender has to protect two things: the ratings the model is trained on, and the items a user fetches once the recommendations arrive. Nudge, the strongest recent system for the first, trains a matrix factorisation across three servers under replicated secret sharing, publishes the item embeddings in the clear, and leaves the fetch to "other means". Pirsona fetches privately but trains with four parties over a Boolean circuit.

We build OblivRec, the composition of the two: Nudge's power-iteration training under 2-of-3 replicated sharing, followed by two-server private information retrieval from a distributed point function, with one implementation of that function serving both the training protocol's comparison gate and the read. On MovieLens-100K, composing costs exactly the sum of the two halves, to the byte. The spec-faithful normaliser, which reveals nothing, costs between 0.0018 and 0.0041 of nDCG@20 against one that reveals a norm, and 5.3 times the rounds. Truncation, not the comparison gate, is where the rounds go. Without the private read, ten observed fetches recover a user's private embedding to a cosine of 0.84, and neither decoy traffic nor Nudge's own differential privacy brings the attack down to what popularity alone achieves.

We also report where the composition is weaker than its parts. On a wide-area link, downloading the whole catalogue beats our unbatched private read by a factor of 4.6. Until we added a one-round check, a malicious client could cast a vote of arbitrary weight in the next model, and sixteen such clients destroyed it. The check stops inflated weight but not a forged key that redistributes it. Security is semi-honest with at most one corrupted server, and it is argued by a simulation sketch, not proven.

Contents

1 Introduction **4**
- 1.1 Two papers, two halves . . . 4
- 1.2 What we built and what we found . . . 4
- 1.3 Organisation . . . 5

2 Threat model and security **5**
- 2.1 Setting . . . 6
- 2.2 Ideal functionalities . . . 6
- 2.3 Adversary classes . . . 6
- 2.4 Leakage profile . . . 6

2.5 A real/ideal simulation sketch . . . 7
2.6 Where the argument does not close . . . 8
2.7 What we do not claim . . . 9

3 Background **9**
3.1 Matrix factorisation by power iteration . . . 9
3.2 Replicated secret sharing . . . 9
3.3 Fixed-point arithmetic and truncation . . . 10
3.4 Function secret sharing . . . 10
3.5 Two-server private information retrieval . . . 10
3.6 Measuring recommendation quality . . . 10

4 System design **11**
4.1 Training . . . 11
4.2 Serving . . . 12
4.3 Delivery . . . 12
4.4 Closing the loop . . . 12
4.5 Design decisions . . . 13

5 Implementation **13**
5.1 The distributed point function . . . 13
5.2 Private serving and delivery . . . 15
5.3 The training substrate . . . 16
5.4 Truncation . . . 16
5.5 The comparison gate and the normaliser . . . 17
5.6 Power iteration . . . 17
5.7 Bugs found by measurement . . . 18

6 Evaluation **18**
6.1 Setup . . . 19
6.2 Recommendation quality . . . 19
6.3 The cost of each half, isolated . . . 20
6.4 The private read against its alternatives . . . 21
6.5 Scaling, and Theorem 4.2 measured . . . 22
6.6 Where the rounds go . . . 22
6.7 How wide the ring must be . . . 24
6.8 On the wire . . . 24
6.9 Server-side timing . . . 25

7 Leakage of the composition **26**
7.1 Why a fetch is a projection . . . 26
7.2 The attack, and the control that makes it a result . . . 26
7.3 Defence one: decoys . . . 28
7.4 Defence two: Nudge's differential privacy . . . 28
7.5 The malicious client . . . 29

8 Related work **31**

9 Limitations **32**

10 Conclusion **34**

A Protocols **37**

B Reproducing every figure and number **40**

1 Introduction

A recommender system is built from the most revealing data its users produce: what they watched, read and bought. That this data is identifying, and not merely sensitive, was shown by Narayanan and Shmatikov [NS08], who matched records in the anonymised Netflix Prize dataset to public profiles and report "uncovering their apparent political preferences and other potentially sensitive information". A recommender that its operator cannot read therefore has to protect more than one thing. It has to protect the ratings the model is trained on, and it has to protect what the user does with the recommendations afterwards, because fetching a recommended item says which item it was.

1.1 Two papers, two halves

The course handed us two systems that each address one of these. Nudge [HDCB26] trains a matrix factorisation of the rating matrix across three servers, at most one of them compromised, under 2-of-3 replicated secret sharing. Its central observation is that a matrix–vector product under this sharing costs communication proportional to the output vector rather than to the matrix, so an algorithm written as a chain of such products with a few non-linear steps between them is cheap. Power iteration has that shape. Nudge publishes the resulting item embeddings $\mathbf{B}$ in the clear, by design, and computes each user's scores in shared form. It then stops, and says so: the system "relies on other means (e.g., Apple's private relay, Tor, or cryptographic private information retrieval) to let users fetch data items in a private way" [HDCB26].

Pirsona [VBH21] addresses the fetch. Users retrieve items by multi-server private information retrieval (PIR), and the servers harvest a secret-shared record of what each user consumed directly from the PIR queries, so the next round of training needs no separate upload. Its training core is a "carefully optimized 4PC Boolean matrix factorization", and its authors are explicit that they "opted for the most performant primitives available (at the expense of rather strong non-collusion assumptions)" [VBH21].

Neither paper builds both halves well, and neither measures what happens when they meet. That meeting point is where this project sits. Nudge's public $\mathbf{B}$ is a complete latent-factor model of the catalogue, known to everyone, so a fetch observed in the clear is not an isolated fact about a user. It is a projection of that user's private embedding onto a basis the observer already holds. Section 7 measures how quickly those projections give the user away.

1.2 What we built and what we found

OblivRec is the composition: Nudge's training core, followed by two-server PIR built from a distributed point function (DPF), with Pirsona's harvest connecting delivery back to training. Everything cryptographic is written from scratch in C++17, including the DPF, which serves both as the read layer and as the comparison gate inside training. Our contributions are these.

1. **A working composition.** Private training runs end to end on MovieLens-100K, the trained model is served from three separate server processes over TCP, and each recommended record is fetched by DPF-PIR and checked byte for byte against a cleartext lookup (Sections 4 and 5).

2. **The cost of the composition, measured half by half.** Composing costs exactly the sum of the halves, which the benchmark asserts to the byte. Nudge's Theorem 4.2 is measured rather than quoted: quadrupling the input matrix raises traffic by 4.4% and leaves the round count unchanged. Truncation accounts for 68.6 to 74.4% of all rounds, and the FSS comparison gate

for 3.1%. A normaliser that reveals nothing costs between 0.0018 and 0.0041 of nDCG@20 and 5.3 times the rounds, and it cannot run in a 64-bit ring at all (Section 6).

3. **Why the fetch must be private, and why cheaper defences fail.** Ten observed fetches recover a user's private embedding to a cosine of 0.84, well above a popularity control. Decoy traffic at five times the real volume leaves the attack far above that control, and NUDGE's own differential-privacy mechanism makes the attack stronger while removing 71% of recommendation quality. We reconcile that figure with NUDGE's own 17% (Section 7).

4. **A malicious-client hole in the composition, and its partial fix.** The harvest path accepted a vote of any weight. We measure what that buys an attacker, close weight inflation with a one-round check, show by construction what the check does not close, and measure the denial-of-service amplification that full-domain evaluation invites (Section 7.5).

5. **A stated security argument.** A threat model with six adversary classes, a leakage profile, and a real/ideal simulation sketch that walks every message. The sketch shows that one of our two normalisers is not simulable against the intended functionality, and we say which numbers came from it (Section 2).

We also report results that go against us, because a reader deciding whether to build this needs them. On a wide-area link, downloading the entire catalogue beats our private read by a factor of 4.6, since delivery costs one round trip per recommendation and we did not batch them. The spec-faithful normaliser loses most of its quality at a single power iteration. And one bug, a silently clamping comparison range, would have been reported as "privacy costs 24% of recommendation quality" had we measured only cost.

What this report does not claim. Security is semi-honest, and two colluding servers learn everything. The evaluation runs on MovieLens-100K, not on MovieLens-1M or Netflix scale. Wide-area profiles are emulated at the channel level, not with kernel network emulation. The security argument is a sketch. Section 9 states each of these in full.

1.3 Organisation

Section 2 fixes the threat model and the security argument. Section 3 covers the primitives, and Section 4 the system design. Section 5 describes the implementation and how it was verified, and Section 6 evaluates its cost and quality. Section 7 measures what the composition leaks and what a malicious client can do. Section 8 places the work, Section 9 lists its limitations, and Section 10 concludes. Appendix A gives the protocols as pseudocode, and Appendix B maps every figure to the command and the data file that produce it.

2 Threat model and security

This section says what OBLIVREC protects, against whom, and how far the argument for it goes. It comes before the design on purpose: every later section is read against it, and the most important sentences in this report are the ones that say what is *not* protected.

2.1 Setting

There are three servers P_0, P_1, P_2, run by independent operators, and m users. The rating matrix $\mathbf{U} \in \mathbb{R}^{m \times n}$ is held in 2-of-3 replicated secret shares, so no single server holds it. Training produces an item-embedding matrix $\mathbf{B} \in \mathbb{R}^{d \times n}$, which is opened to everyone, and user embeddings $\mathbf{A} \in \mathbb{R}^{m \times d}$, which stay shared. Opening $\mathbf{B}$ is NUDGE's design and not an oversight: each converged row is revealed before the next is computed, which is what keeps orthogonalisation cheap.

To be served, user i asks each server for its share of the score vector $\mathbf{a}^{(i)}\mathbf{B}$, reconstructs it, and selects the top k items *on the user's own machine*. The user then fetches each of the k records by two-server DPF-PIR against P_0 and P_1. The catalogue of records is public, and all three servers hold it.

2.2 Ideal functionalities

Two functionalities describe what the system is meant to compute and to whom each output goes. They are taken from the project's requirements document.

$\mathcal{F}_{\text{train}}([\![\mathbf{U}]\!], d, \ell)$: $(\mathbf{A}, \mathbf{B}) \leftarrow$ ApproxFactor($\mathbf{U}$) by ℓ power iterations per component,
every server receives $\mathbf{B}$ in the clear and its shares of $[\![\mathbf{A}]\!]$.

$\mathcal{F}_{\text{serve}}([\![\mathbf{a}^{(i)}]\!], \mathbf{B}, k)$: $\mathbf{s} \leftarrow \mathbf{a}^{(i)}\mathbf{B}$, $\quad \mathbf{s}[j] \leftarrow -\infty$ for every item j user i has rated,
$T \leftarrow$ the top-k indices of $\mathbf{s}$,
the user receives T and the records $D[T]$, and the servers receive nothing.

The second line of $\mathcal{F}_{\text{serve}}$'s output is what OBLIVREC adds to NUDGE. A third functionality is needed because of one of our two normalisers, and Section 2.6 explains why: $\mathcal{F}_{\text{train}}^{\text{leaky}}$ is $\mathcal{F}_{\text{train}}$ with the sequence of norms $\{\|\mathbf{v}\|\}$ from every normalisation added to the servers' output.

2.3 Adversary classes

Table 1 lists six classes. The headline guarantee is against **A1**, one semi-honest server, which is exactly NUDGE's model.

2.4 Leakage profile

Hidden from any single server. Individual ratings, and which items a user rated. The user embeddings $\mathbf{A}$. Each score vector, and the top-k indices chosen from it. Which records a user fetches.

Leaked by design. The parameters m, n, d, ℓ, k, b, t. The item-embedding matrix $\mathbf{B}$, in the clear. The number of non-zero ratings, which the truncation schedule uses as a public bound and which NUDGE also leaks, unless every user is made to submit a fixed number of ratings at a cost in quality. How many queries each user makes, and when.

Outside the protocol's protection. Correlation between when a user queries and external events. Anything an adversary can infer from $\mathbf{B}$ together with an observed fetch, which is precisely why the fetch has to be private and is the subject of Section 7.

Table 1: Adversary classes and what OblivRec guarantees against each. A3 is the assumption the whole system rests on.

	Adversary	Guarantee
A1	One semi-honest server	Nothing is learned beyond the leakage profile of Section 2.4. Argued by the sketch in Section 2.5.
A2	One semi-honest server and colluding users	Honest users' ratings stay hidden. Model *quality* is not protected: colluding clients can distort the harvested training input (Section 7.5).
A3	Two or more colluding servers	**No guarantee.** Any two servers hold all three shares and reconstruct everything, and the two PIR servers together reconstruct every fetched index.
A4	A malicious (deviating) server	Out of scope, as in Nudge. A deviating server can corrupt correctness and availability.
A5	A malicious user	Retrieval corrupts only that user's own output. The harvest path accepted a vote of any weight, which we close for inflation but not for redistribution, and full-domain evaluation is an unmitigated denial-of-service lever (Section 7.5).
A6	A network observer	Sees message sizes and timing. Sizes depend only on public parameters, measured in Section 6.8. Server-side timing is bounded in Section 6.9, and wire timing is not analysed.

2.5 A real/ideal simulation sketch

The claim, for class A1, is that for each corrupted server P_c there is a simulator $\mathcal{S}_c$ which, given only P_c's input shares and the output the functionality declares for it, produces a transcript computationally indistinguishable from P_c's real view. That view is its input shares, its pseudorandom-function keys, its local randomness, and every ring element it receives. We argue it step by step against the code. This is a *sketch*. There is no formal security parameter, no reduction, and no written hybrid argument, and the three servers are not symmetric, since only P_0 and P_1 hold DPF keys and the truncation protocol rotates a helper role. So "by symmetry" is never used.

The substrate. A value is shared as $x = x_0 + x_1 + x_2$ with party i holding (x_i, x_{i+1}). Any two of three shares of a 3-out-of-3 additive sharing are uniform and independent of x, so $\mathcal{S}_c$ samples two uniform ring elements per value and never needs $\mathbf{U}$. Correlated randomness comes from pairwise PRF keys. Party i derives $\alpha_i = F(k_i, ctr) - F(k_{i-1}, ctr)$, and the three sum to zero with no communication. P_c can compute its own α_c but no other party's, because each needs a key it does not hold. That asymmetry is what makes every incoming element below uniform.

Matrix–vector products. Each party computes its share of the product locally, masks it with α_i, and sends one ring element per output entry. P_c receives z_{c+1}, masked by α_{c+1}, which uses a key P_c lacks, so $\mathcal{S}_c$ samples it uniformly. The message size is set by the public output length alone.

Truncation. A helper h supplies a correlated pair $(r, r \gg t)$ and the other two parties open $c = x - r$ and reshare. There are two cases. If P_c is an opener, it sees $x - r$ with r uniform and known only to h, so $\mathcal{S}_c$ samples c uniformly. If P_c is the helper, it sees only the pair it generated, and the reshare is masked with a pairwise secret it does not hold, so $\mathcal{S}_c$ replays its own randomness and samples the rest. A party that held r and also saw $x - r$ would recover x, which is why the helper is passed explicitly at every call site.

Normalisation. For the spec-faithful `FssNormalizer`, the comparison gate opens $S + r$ where r is drawn from a range $\kappa = 40$ bits wider than S, so the opening is statistically close to uniform and $\mathcal{S}_c$ samples it. The DCF evaluations are local, and the Newton iterations are multiplications and truncations already covered.

Orthogonalisation. Entirely local, because every product is a share times a public row of $\mathbf{B}$. Nothing is sent.

Opening each row of B. $\mathbf{B}$ is $\mathcal{F}_{\text{train}}$'s declared output, so $\mathcal{S}_c$ is given it and replays it. This step is free in the simulation and is also the largest leakage surface in the system. That is not a contradiction. It is leakage by design, and Section 7.2 is the measurement of what it costs.

Serving. Scores are a public matrix applied to a shared vector, so every server computes its share locally and sends it to the user, not to another server. The seen-item mask arrives from the user as a share, which $\mathcal{S}_c$ samples.

Delivery. $P_c \in \{P_0, P_1\}$ receives one DPF key per fetch. A single key is pseudorandom and independent of the index by the DPF's security, so $\mathcal{S}_c$ samples a key of the right length, 227 bytes at 11 domain bits. The server then expands the key over the whole domain and takes an inner product with every record, so the work does not depend on the index. P_2 receives nothing in this step.

Harvesting. The difference $h_0 - h_1$ of the two PIR servers' accumulated expansions is a 2-of-2 sharing of the consumption counts. It is converted to 2-of-3 replicated form after re-randomising with a pairwise secret that P_2 does not hold, so $\mathcal{S}_c$ samples uniformly. Without that re-randomisation, P_2 would be sent h_0 in the clear alongside a zero share and could subtract its way to the counts.

2.6 Where the argument does not close

1. The revealing normaliser is not simulable against $\mathcal{F}_{\text{train}}$. The cheaper normaliser opens $\|\mathbf{v}\|^2$ at every normalisation, which is $d(\ell + 1)$ scalars per run: the trajectory of the power iteration's norms, which depend on the singular values of $\mathbf{U}$. No simulator given only $\mathbf{B}$ can produce them, because they are not a function of $\mathbf{B}$. We could exclude this path and claim security only for the spec-faithful one. We instead claim security for it against the weaker $\mathcal{F}_{\text{train}}^{\text{leaky}}$, because that is what we ran: **every $b = 64$ quality number in this report comes from the revealing path**. Claiming the strong statement while reporting numbers from the weak one would be the dishonest version of this section. Section 6.2 gives the price of the path that does close.

2. Repeated training is argued, not proven. Each round is covered above, and harvesting adds no leak per round. What an adversary learns from many successive openings of $\mathbf{B}$, each trained on consumption that the previous round's recommendations influenced, is not answered here.

3. Composition is sequential and assumed. The protocols run one after another with fresh correlated randomness, the standard setting for this kind of argument, but no composition theorem is invoked and no universal-composability claim is made.

2.7 What we do not claim

We claim nothing against two colluding servers, against a deviating server, or against an observer who sees both PIR links at once, who reconstructs the record by the definition of two-server PIR. We do not claim constant-time execution, only a bound on index-dependent server time. We do not claim a proof. And we do not claim that hiding the fetch hides the recommendation itself from anyone who later sees what the user does with it.

3 Background

This section fixes the primitives the rest of the report is built from. It is deliberate about *costs*, and in particular about rounds, because on a wide-area link the round count decides the running time of every protocol here.

3.1 Matrix factorisation by power iteration

Collaborative filtering predicts a user's preferences from the preferences of other users, and matrix factorisation is its dominant latent-factor technique [KBV09, HKV08]. Given the rating matrix $\mathbf{U} \in \mathbb{R}^{m \times n}$, it seeks $\mathbf{U} \approx \mathbf{AB}$ with $\mathbf{A} \in \mathbb{R}^{m \times d}$, $\mathbf{B} \in \mathbb{R}^{d \times n}$ and $d \ll \min(m, n)$. The predicted affinity of user i for item j is the inner product of their embeddings, and a user's recommendations are the k highest-scoring items the user has not already rated.

Nudge computes the factorisation by power iteration rather than by gradient descent. For each of d components it starts from a random vector $\mathbf{v}$, and ℓ times it multiplies by $\mathbf{U}^\top\mathbf{U}$, projects out the components already found, and normalises. The converged $\mathbf{v}$ becomes a row of $\mathbf{B}$, and finally $\mathbf{A} = \mathbf{UB}^\top$. Power iteration on $\mathbf{U}^\top\mathbf{U}$ converges to the top-d right singular subspace of $\mathbf{U}$, at a rate set by the gap between consecutive singular values. What makes this formulation attractive under secret sharing is its shape: almost all of the work is matrix–vector products, with one normalisation per step.

3.2 Replicated secret sharing

In 2-out-of-3 replicated sharing over the ring $\mathcal{R} = \mathbb{Z}_{2^b}$, a value x is written as $x = x_0 + x_1 + x_2$, and party P_i holds the pair (x_i, x_{i+1}) with indices taken modulo three [AFL+16]. Any two parties can reconstruct x, while a single party holds two uniformly random summands and learns nothing. We write $[\![x]\!]$ for a sharing of x. Addition, and multiplication by a public constant, are local.

Multiplication of two shared values is where the structure pays off. Because each party holds two of the three summands of each factor, it can compute three of the nine cross terms of xy on its own, and between them the three parties cover all nine. Each party therefore produces an additive share z_i of xy locally. To return to replicated form, each party masks z_i with a share of zero and sends it to one neighbour: one round and one ring element per party per product. The zero shares cost no communication in the *PRF model*, where each pair of parties holds a shared pseudorandom-function key and the three parties derive $\alpha_i = F(k_i, ctr) - F(k_{i-1}, ctr)$, which sum to zero [AFL+16].

The consequence Nudge builds on is its Theorem 4.2. An inner product is a sum of products, and the sum can be taken before resharing, so a whole inner product costs one ring element per party, not one per term. A matrix–vector product is one inner product per output entry. Its communication is therefore proportional to the length of the *output* vector, whatever the size of the matrix.

3.3 Fixed-point arithmetic and truncation

Recommendation works over the reals and the sharing works over a ring, so a real v is encoded as the integer $\lfloor v \cdot 2^t \rceil \in \mathcal{R}$ with t fractional bits. Addition then works unchanged, but the product of two values at scale t is at scale $2t$ and must be *truncated* back, or the scale grows until the ring overflows. Truncating shares naively, by having each party shift its own share, fails: the shares are uniform over the ring, so their sum wraps with high probability, and a wrap turns into an error of 2^{b-t} in the result. Correct truncation needs an interactive protocol. It is the most frequent non-linear step in fixed-point secure computation, and prior work treats it as a first-class cost [MZ17, EGK+20]. Section 6.6 shows that it dominates ours.

3.4 Function secret sharing

Function secret sharing splits a *function* rather than a value [BGI15, BGI16]. A two-party scheme turns $f : \{0,1\}^D \to \mathcal{R}$ into keys k_0, k_1 such that each key alone hides f and the two evaluations recombine to $f(x)$ at every x. A *distributed point function* (DPF) is the case where $f = f_{\alpha,\beta}$ is β at $x = \alpha$ and zero elsewhere [GI14]. The construction of Boyle, Gilboa and Ishai is a binary tree of pseudorandom seeds, and each key holds one seed and one correction word per level [BGI16]. Keys are therefore *logarithmic* in the domain size 2^D, while evaluating the whole domain costs linear work, a trade made concrete in Section 5.1.

A *distributed comparison function* (DCF) shares the step function $f_{<\alpha}(x) = \beta \cdot 1[x < \alpha]$ in the same way [BCG+21]. A DCF key hides its threshold and is evaluated at a public point. Secure computation needs the opposite, a secret input compared against a public threshold, and Boyle et al. obtain it by masking. A dealer draws a random r and builds keys for the threshold shifted by r. The parties open $c = x + r$, which reveals nothing about x if r is drawn from a wide enough range, and evaluate their keys at c. The evaluation yields shares of the comparison on x [BCG+21]. Every threshold evaluated at the same opening costs no further round.

3.5 Two-server private information retrieval

Private information retrieval lets a client fetch record α of a database D without the server learning α [CGKS95]. With two servers that do not collude, a DPF gives a simple scheme [BGI15, HH19]. The client generates keys for $f_{\alpha,1}$ and sends one to each server. Server c expands its key over the whole domain and answers with

$$\mathrm{ans}_c \;=\; \sum_j \mathrm{EvalFull}(k_c)[j] \cdot D[j],$$

and because the two expansions recombine to the indicator of α, the two answers recombine to $D[\alpha]$. The upload is one logarithmic-size key per server. The price is on the server side: each answer touches every record, and it must, because a server that skipped records would learn which ones the client could not have wanted.

3.6 Measuring recommendation quality

We report normalised discounted cumulative gain at 20, nDCG@20, over the held-out ratings of MovieLens-100K's predefined u1 split [HK16], with an item counted as relevant when its held-out rating is at least four and with items already rated in training excluded from the ranking. The split has 943 users and 1,682 items. Our metric was cross-checked against an independent implementation

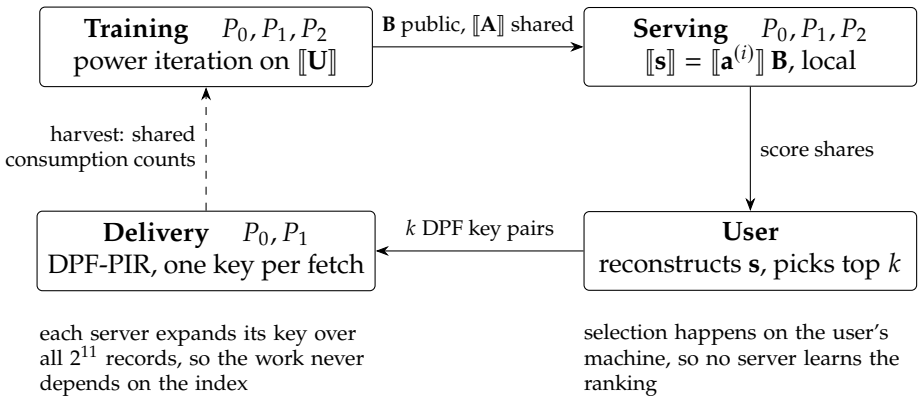


Figure 1: The OblivRec cycle. Solid arrows run in every experiment in this report. The dashed harvest arrow is implemented and tested as a library path (Section 4.4) but is not yet driven by the networked server binaries, and every training run here reads its ratings from disk.

on two hundred random instances. Because the relevance convention differs, our absolute values are a within-project baseline and are not comparable to the numbers Nudge reports on Netflix data.

4 System design

OblivRec has three stages that form a cycle (Figure 1). Training runs periodically over all users and produces the model. Serving and delivery run per user, online. The harvest carries what users fetched back into the next round of training.

4.1 Training

Training takes $[\![\mathbf{U}]\!]$ and runs Nudge's ApproxFactor under replicated sharing: for each component, ℓ rounds of two shared matrix–vector products, a projection against the rows of $\mathbf{B}$ already found, and a normalisation. Appendix A gives it as Algorithm 1. Three pieces of it need design decisions of their own.

Where truncations go. Ratings are integers, held at scale zero, so the two matrix products need no truncation of their own, only a fixed rescaling shift afterwards that keeps $\|\mathbf{v}\|^2$ from overflowing. The remaining truncations sit in orthogonalisation and normalisation, and each costs a three-round protocol. Orthogonalisation can be *deferred*: subtract the projections at scale $3t$ and truncate once by $2t$. Or it can be *immediate*: truncate the inner products first and the result afterwards, two truncations that never exceed scale $2t$. Deferral saves a truncation and costs t bits of headroom, and whether that fits depends on b, t, n and d. So OblivRec derives the schedule from public parameters at start-up, proves it fits with at least eight spare bits, and refuses to run otherwise. At $b = 64$ on MovieLens-100K the deferred schedule does not fit and training uses the immediate one, with 21 bits to spare. At $b = 128$ the deferred schedule fits with 67.

How to normalise. Normalising needs $1/\|\mathbf{v}\|$ on a shared $\mathbf{v}$. OblivRec has two implementations behind one interface. The spec-faithful one follows Nudge: a DCF-based gate finds the most significant non-zero bit of $\|\mathbf{v}\|^2$, which gives a first approximation to $1/\|\mathbf{v}\|$, and Newton–Raphson

refines it. It reveals nothing. The second opens $\|\mathbf{v}\|^2$, divides in the clear, and rescales by a public constant. It is far cheaper and it leaks, and Section 2.6 states exactly what that costs the security argument. Every result records which normaliser produced it.

Keeping B public. Because each finished row of $\mathbf{B}$ is opened, projecting against it is a product of shares with public values, which needs no multiplication protocol. Our implementation priced the alternative: keeping $\mathbf{B}$ shared makes one small orthogonalisation cost 6 rounds and 1,800 bytes where the public version costs none. We keep Nudge's choice, and Section 7 measures its consequence.

4.2 Serving

Each server holds a share of the user's embedding $\mathbf{a}^{(i)}$ and the public $\mathbf{B}$, so it computes its share of all n scores locally, with no communication between servers. The user also supplies a shared mask carrying a large negative sentinel at the items it has already rated. The servers add it to their shares, which is again local, and send the result to the user, who reconstructs the score vector and selects the top k.

The design draft specified an oblivious top-k selector run by the servers. We cut it: once the user reconstructs the scores, no server learns the selection whether or not an oblivious selector exists, so the selector would protect something already protected. The cost is that the user downloads a score share of length n from each server, which Section 9 returns to.

4.3 Delivery

Records are packed to a fixed width of $L = 256$ bytes, since a variable-width record would leak its identity through the reply length. MovieLens-100K's 1,682 items pad to a domain of $2^{11} = 2{,}048$ entries, and padding records are all zero, which leaks nothing because they are the same width as real ones. The DPF is a two-party primitive, so P_0 and P_1 answer PIR queries and P_2 does not. All three servers hold the catalogue, which is public. For each of its k recommendations the user sends one key to each PIR server, and the difference of the two answers is the record.

4.4 Closing the loop

When a PIR server expands a key, the expansion it computes is its share of the indicator vector of the fetched item. Pirsona's observation is that this is exactly a secret-shared record of consumption, already computed and normally thrown away. OblivRec's answer routine returns the expansion alongside the answer, an accumulator sums it over queries, and a one-round conversion turns the two PIR servers' 2-of-2 shares into the 2-of-3 replicated form training consumes. A further one-round check, added after Section 7.5 found the need for it, confirms that each query carries weight exactly one. Algorithm 4 gives both.

We state the extent of this plainly. The harvest, the conversion and the weight check are implemented, and a test drives real PIR queries through them and multiplies the result under the training substrate. The networked server binaries do not yet accumulate expansions across sessions or retrain from them, and every training run in this report reads MovieLens ratings from disk.

Table 2: The design decisions that matter most, from the project's decisions log.

Decision	Reason
Power iteration, not gradient descent	Matrix–vector products cost communication in the output length only, so only truncation and normalisation are expensive. This is Nudge's core idea.
Three servers, 2-of-3 replicated sharing, semi-honest	Matches Nudge exactly, one fewer non-colluding party than Pirsona's four.
B opened row by row	Makes orthogonalisation local. The cost is the leakage measured in Section 7.
One DPF implementation for both halves	The same primitive is the PIR query and, through the DCF built beside it, the comparison gate. Forking it would double the code most likely to be wrong.
Top-k on the user's machine	The user reconstructs the scores anyway, so a server-side oblivious selector would protect nothing further.
Fixed-width records	A variable width leaks the record through the reply length.
Ring width a template parameter	Whether $b = 64$ suffices was an open question, and the answer turned out to depend on which normaliser runs (Section 6.7).
Three OS processes, no threads	Each server serves one client to completion, so nothing is concurrent, and separate processes make that true rather than likely.

4.5 Design decisions

Table 2 collects the decisions that shape everything else, each with the reason recorded when it was taken.

5 Implementation

OblivRec is written in C++17 with no cryptographic library. It comprises about 7,400 lines of library and application code and about 3,300 lines of tests in eighteen test binaries, with Python for the cleartext oracle, the export of fixed-point models and the leakage studies. The pseudorandom generator is fixed-key AES-128 in Davies–Meyer mode through AES-NI instructions, and key material comes from the operating system's generator. Table 3 maps the code. The repository also vendors Nudge's Go reference implementation under `third_party/`, under its MIT licence, as a reference to read alongside the paper. No build target compiles or links it.

5.1 The distributed point function

The DPF follows Boyle, Gilboa and Ishai [BGI16]. It is written by us rather than imported because it is the component both halves depend on, and a viva will probe it hardest.

Key size. A key holds a header, an initial seed, one correction word of 18 bytes per level, and a final correction word of $\lceil b/8 \rceil$ bytes, so its size is $1 + 4 + 16 + 18D + \lceil b/8 \rceil$ bytes for a domain of D bits. The serialiser's reported size is asserted equal to the length it actually writes. At the 11 domain bits MovieLens-100K needs, a key is 227 bytes against 16,384 bytes for a naive one-hot query, and the gap widens without bound (Figure 2). Measuring it corrected the design draft, which had estimated 260 bytes at 4,096 items. The measured value is 245 bytes, and the draft's formula had omitted the seed and the header while evaluating at the wrong domain width.

Table 3: Where each part of the system lives.

Directory	Contents
`src/dpf/`	The DPF: tree generation, single-point and whole-domain evaluation, key serialisation, and the DCF built beside it.
`src/mpc/`	Replicated sharing, pairwise-PRF zero shares, shared matrix–vector products, truncation, the MSNZB comparison gate and the shared inverse square root.
`src/mf/`	Power iteration under sharing, orthogonalisation, both normalisers, the truncation schedule, and a cleartext fixed-point twin.
`src/serve/`, `src/pir/`	Score shares and the seen-item mask. The catalogue, the PIR server and client, and the harvest.
`src/net/`	Length-prefixed TCP framing, the transcript recorder, and channel-level network emulation.
`src/apps/`	The three servers, the training driver, the end-to-end demo, and the probe used by the distinguisher.
`model/`	The cleartext oracle, the float-to-ring export, the reconstruction attack, and the differential-privacy and poisoning studies.
`bench/`	Seven benchmarks writing one JSON row per measurement, and the script that draws every figure from those rows.

Sign convention. Textbook presentations negate party one's output, which gives the sum $\mathrm{Eval}(k_0, x) + \mathrm{Eval}(k_1, x) = f(x)$. Our specification states the invariant as a difference, so we omit the negation and have $\mathrm{Eval}(k_0, x) - \mathrm{Eval}(k_1, x) = f(x)$ at every interface, in the PIR read and in the harvest. The DCF uses the textbook convention internally, as its construction is defined, and converts at its boundary. Getting the convention backwards makes every test fail in a way that looks like a traversal bug rather than a sign error, so it is recorded at every use.

Key material. The two initial seeds are the entire secret. The first version seeded them from a Mersenne Twister, which is a genuine break: that generator is reconstructible from 624 consecutive outputs, so an adversary who saw enough key material could rebuild the tree and recover α. Seeds now come from the operating system's generator, and the failure policy is to abort rather than fall back to a weaker source.

Exhaustive verification. The characteristic failure of this construction is a wrong control-bit update, which stays invisible until enough levels accumulate and then presents as "correct at two domain bits, wrong at eight". So the tests walk both parties' trees in lockstep and assert the structural invariant at every internal node: on the path to α the seeds differ and the control bits disagree, and off it both are equal. The difference invariant itself was then checked for every α against every x in every domain up to 16 bits on the 64-bit ring and 15 bits on the 128-bit ring, which is 7.16×10^9 leaf comparisons. That sweep passed at commit `bce4cb1`. The default test target repeats it up to 11 bits on both rings in about a second, and β is derived from the domain and α so that billions of comparisons do not all exercise the final correction word at one value.

A bug found by testing the wire format. Deserialisation accepted a party byte outside $\{0, 1\}$. Evaluation starts its control bit at the party index and branches on whether it is non-zero, so a party byte of 7 made the key *silently evaluate as party one*. Keys arrive over the network, so the party byte

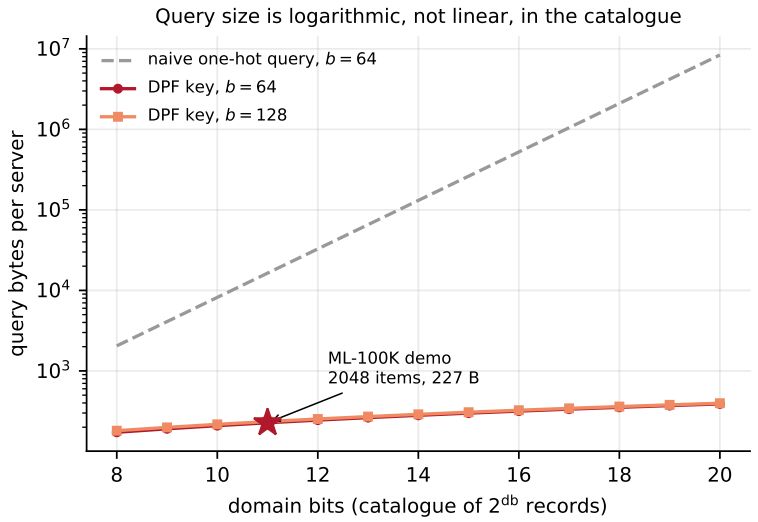


Figure 2: Query size against catalogue size. The DPF key grows by 18 bytes per domain bit while the naive query doubles. The star is the operating point of every experiment in this report. The plotted closed form is the one `tests/test_dpf_serialize.cpp` asserts equal to the serialised length. Generated by `mingw32-make figures`.

and the domain width are now validated before either is used, and the tests that found the bug are kept as regression tests.

5.2 Private serving and delivery

The catalogue. MovieLens-100K's item file is variable-length text, so each record is packed into 256 bytes with explicit one-byte length prefixes rather than delimiters. The file contains Latin-1 bytes and does not decode as UTF-8, so it is read and carried as raw bytes. The longest record uses 117 bytes, and every record is checked against the budget at load time, so a longer title fails loudly at start-up rather than being cut at answer time.

The read. Each PIR server expands its key over all 2,048 indices and takes an inner product with the catalogue, viewed as 32 ring words per record at $b = 64$. Two properties carry the obliviousness claim. The loop visits every record in index order with no branch on the data, and every expanded coefficient is a pseudorandom share, so there is no sparse pattern for a compiler, a cache or a timing observer to find. Reads are checked byte for byte against a cleartext lookup at 60 random indices on the 64-bit ring and 20 on the 128-bit ring, including both ends of the domain and padding slots.

Scores. Both factors of a score are encoded at $t = 20$ fractional bits, so scores are at scale $2t$, and serving decodes at that scale instead of truncating, because truncation is interactive. Getting this wrong would scale every score by 2^{20}, which is monotone and therefore invisible in a ranking, so it is recorded explicitly. The seen-item sentinel is -2^{55}, about 3,700× below the largest real score and 256× above the bottom of the ring, both asserted. The top-k comparison is signed, because comparing two's-complement values as unsigned would sort the masked items first.

The float-to-ring boundary. One program converts floats to ring elements, and a committed file of 25 value pairs checks it against the C++ side on every test run. It found two bugs on its first run. Python's `rint` rounds half to even while C++'s `round` rounds half away from zero, so the two disagreed at every exact half-quantum. And the reference scores had been computed by quantising the float product, while the protocol multiplies quantised factors, so the two disagreed on 1,650 of 1,682 scores by about 5×10^{-6} relative. That is irrelevant to the ranking, which is exactly why a tolerance-based check would have accepted it. Serving is now checked against the oracle by exact equality, and all 1,682 scores match.

Three processes. The servers are three operating-system processes that exchange length-prefixed frames over TCP. TCP is a byte stream, so one `send` can arrive as several `recv` calls. The test forces this with a deliberately small receive buffer and checks that a 16,384-byte frame arrives as one frame. The declared frame length is attacker-chosen and is checked against a cap before it sizes any allocation. Every frame can be recorded at the channel boundary into a transcript, which Section 6.8 analyses.

5.3 The training substrate

Sharing and zero shares. `Mpc3` holds all three parties' state and moves every inter-party message through one exchange routine, which counts rounds and bytes. That counter is what every cost in Section 6 is read from, so a protocol cannot send without paying. Zero shares come from three pairwise PRF generators.

The counter invariant. The three generators must advance in lockstep. A protocol that draws masks for only two of the three parties leaves the third generator one step behind, and every later zero share then fails to sum to zero. Nothing crashes: the parties go on producing well-formed shares of the wrong value. This happened once and silently corrupted every subsequent multiplication. The exchange routine now asserts that all three counters agree, and protocols in which one party draws nothing advance its generator explicitly.

Theorem 4.2, asserted. The tests check Nudge's central cost claim against the byte counter rather than in prose: multiplying a shared 4×64 matrix by a shared vector costs one round and 96 bytes, which is three parties sending four 8-byte elements each, set by the four outputs and not by the 256 matrix entries.

5.4 Truncation

`TruncatePair` is the standard correct construction. A helper party draws r and gives the other two parties shares of r and of $r \gg t$. Those two open $x - r$ between themselves, shift it in the clear, add back their shares of $r \gg t$, and reshare under a pairwise mask the helper cannot compute (Algorithm 2). It costs three rounds and eight ring elements per value. Over 3,200 random values its error is never more than one unit in the last place. The naive alternative, where every party shifts its own shares, costs nothing and is catastrophically wrong on about a quarter of values in our tests, because the shares wrap the ring. That comparison is the whole argument for paying three rounds, and Section 6.6 shows the three rounds are most of what training costs. The helper is a parameter at every call site, because a helper that also saw the opening would learn x.

5.5 The comparison gate and the normaliser

The DCF. The design draft had asserted that a DPF and a DCF are the same primitive. They are not, and NUDGE's reference implementation keeps them in separate modules. A DCF is non-zero on a whole prefix of the domain, so every node of its tree carries a value word as well as a seed, and every level needs a value correction word that depends on everything below it. We built it beside the DPF, sharing the pseudorandom generator, and verified it the same way: every threshold against every input, exhaustively up to 7 domain bits on the 64-bit ring and 6 on the 128-bit ring.

The MSNZB gate. The normaliser needs the most significant non-zero bit of a shared value $S = \|\mathbf{v}\|^2$. The gate uses the mask-and-reveal technique of Section 3.4. Offline, a dealer draws r and builds one DCF key pair per threshold 2^k, shifted by r. Online, the parties open $c = S + r$ and evaluate every key at c. Since $\sum_k 1[S \geq 2^k]$ counts the thresholds below S, any step function of $\mathrm{msnzb}(S)$ is a public linear combination of those indicators. The seed for $1/\|\mathbf{v}\|$, a power of two within a factor of $\sqrt{2}$ of it, is one such function. All thresholds share one opening, so the gate costs two rounds, one to open and one to reshare, however many thresholds it covers. Algorithm 3 gives the gate and the Newton iteration it seeds.

Why the gate needs a 128-bit ring. The opening $c = S + r$ hides S only if r comes from a range κ bits wider than S's. With $\kappa = 40$ and the values power iteration produces, the DCF domain needs 81 bits. A 64-bit ring has no room for the mask at all, so `FssNormalizer` refuses to construct at $b = 64$ rather than masking with too few bits, since a gate that did so would look like privacy while providing none.

Inverse square root. `InvSqrtShared` takes the gate's seed and refines it with Newton–Raphson. Four steps reach a worst relative error of 1.41×10^{-6} over our test values, for 50 rounds in total. The step count was chosen from a cleartext reference first, which measured the error at every step count before any shared code existed.

The gate cannot check its own range. The gate covers thresholds from 2^{lo} to 2^{hi}. A value outside that range does not raise an error. It clamps, because the answer is a telescoping sum over the thresholds. And the gate cannot check, because it only ever sees $S + r$, and learning $\mathrm{msnzb}(S)$ is precisely what it exists to avoid. So the caller must size the range from public parameters. Section 5.7 describes what happened when a caller did not.

5.6 Power iteration

`ApproxFactorShared` runs Algorithm 1 with the truncation schedule and either normaliser. On a small test instance it agrees with its cleartext fixed-point twin to 1.5×10^{-6} in $1 - |\cos|$ per row, with every row of $\mathbf{B}$ unit length to 3.2×10^{-6}. The start-up headroom assertion is not decorative. It refuses $t = 30$ at $b = 64$, and it caught a real overflow: $\|\mathbf{v}\|^2$ of a vector not yet normalised needs 84 bits, which led to the fixed rescaling shift applied right after the two matrix products.

The training driver writes $\mathbf{B}$ and shares of $\mathbf{A}$ to disk, and the demo serves them. `demo` with the privately trained, spec-faithful model at $b = 128$ returns ten film titles for user 42, headed by *Toy Story*. Each record is byte-exact against a cleartext lookup, and the ranking agrees position for position with an independently computed reference. That run is the composition end to end: a

Table 4: Defects found by tests and measurements rather than by inspection.

Defect	How it was found	What it would have caused
DPF party byte unvalidated	serialisation tests	a malformed key silently evaluates as the other party
Rounding mode differs across the Python and C++ boundary	25-vector contract test	off-by-one ring elements at every half-quantum
Reference quantised the product, protocol the factors	exact score comparison	1,650 of 1,682 scores wrong, invisible to a tolerance check
PRF counters out of step	multiplication tests	well-formed shares of the wrong value, every product after the first
$\|\|\mathbf{v}\|\|^2$ overflows before normalisation	start-up headroom assertion	silent wrap of the ring, a meaningless model
MSNZB gate range too narrow	quality study on the no-leak path	0.108 of nDCG@20 lost at $\ell = 10$, with identical cost
Transcript writers interleaved	parsing the transcript	invalid JSON in the evidence file

model no server saw in the clear, served from shares, delivered without either PIR server learning what was fetched.

5.7 Bugs found by measurement

The project's rule was that every private component is checked against a cleartext twin, and every approximate protocol has its error measured rather than assumed. Table 4 lists what that found. None of these would have been found by reading the code, and most produced no error at all.

The gate-range bug deserves its own paragraph, because it is the one that nearly became a finding. The training driver first called the MSNZB gate with the range $[t - 8, t + 8]$, copied from the parameter sweep. Values outside it clamped, Newton–Raphson started from the wrong power of two, and the power iteration never converged: it ended orthogonal to the oracle's subspace. At $\ell = 10$ the no-leak model scored 0.3428 against 0.4513 for the revealing path, which would have been reported as "revealing nothing costs 24% of recommendation quality". The range is now $[1, 2t + 8]$ and the gap is 0.0020. Rounds and bytes are *identical* under both ranges, because a wider range buys more DCF keys offline while the online gate stays two rounds, which is why the cost benchmark that first used the gate could never have caught it. The rows from the buggy run were not kept, so the two nDCG values above are recorded in the commit that fixed it (`d2138ad`) and in the comment at the gate's call site, not in the committed data.

6 Evaluation

This section asks what private training and private delivery each cost, what they cost together, where the cost goes, and what it does to recommendation quality. It reports two results that go against the system in the body rather than in a footnote.

Table 5: nDCG@20 against the number of power iterations ℓ, at $d = 16$. The $b = 128$ revealing column is the control that isolates the normaliser: it shares the spec-faithful path's ring and truncation schedule and differs only in revealing the norm. The last column is the no-leak path's loss against that control. Data: `bench/results/train_quality.jsonl`.

ℓ	reveal, $b = 64$	reveal, $b = 128$	**no leak**, $b = 128$	oracle	no-leak cost
1	0.3729	0.3727	**0.1651**	0.3664	0.2076
2	0.4251	0.4248	0.4230	0.4220	0.0019
5	0.4485	0.4490	0.4448	0.4455	0.0041
10	0.4511	0.4513	0.4493	0.4536	0.0020
25	0.4532	0.4531	0.4499	0.4545	0.0031
50	0.4531	0.4532	0.4502	0.4525	0.0030

6.1 Setup

All measurements use MovieLens-100K's `u1` split (943 users, 1,682 items, 80,000 training ratings) at $d = 16$, $\ell = 10$ and $t = 20$, with $b = 64$ and the revealing normaliser unless stated. Delivery fetches $k = 10$ records per session, and quality is nDCG@20. The machine is a Windows 11 laptop, and the code is built by MSYS2's g++.

Network profiles. We use four: `local`, `lan` (1 ms round trip, 1 Gbit/s), `wan_a` (30 ms, 100 Mbit/s) and `wan_b` (100 ms, 10 Mbit/s). The development machine has neither Docker nor kernel network emulation, so the profiles are applied at the channel level. Delivery runs over real sockets wrapped in a channel that delays each frame. Training runs its three parties in one process, so its time on a profile is computed from the exactly counted rounds and bytes as rounds times round-trip time plus bytes over bandwidth. We call this channel-level emulation throughout and never kernel emulation.

What to trust. Round and byte counts are exact and deterministic. Absolute timings on this laptop vary by up to a factor of four between back-to-back runs, while repetitions within one run agree closely, which is the pattern of thermal throttling. So we quote ratios taken within one run, and where a number derived from absolute times moves between runs we give the range we observed. Every figure is drawn by `mingw32-make figures` from the newest commit's rows in the named data file, and Appendix B maps each one to the command that produces it.

6.2 Recommendation quality

Table 5 compares private training with the cleartext oracle, which runs the same power iteration in floating point. The revealing path at $b = 64$ stays within 0.007 of the oracle at every ℓ. Fixed-point arithmetic and a real truncation protocol cost essentially nothing in quality.

Revealing nothing costs little quality. For every $\ell \geq 2$ the spec-faithful normaliser loses between 0.0018 and 0.0041 of nDCG@20 against either revealing column. The $b = 128$ revealing column is what makes that claim defensible. Without it, the comparison would confound the normaliser with the ring width and the deferred truncation schedule that $b = 128$ allows. The two revealing columns differ by at most 0.0005, so the normaliser accounts for the whole gap. What revealing nothing costs instead is rounds: 11,519 against 2,191 at $\ell = 10$ and $b = 128$, a factor of 5.3.

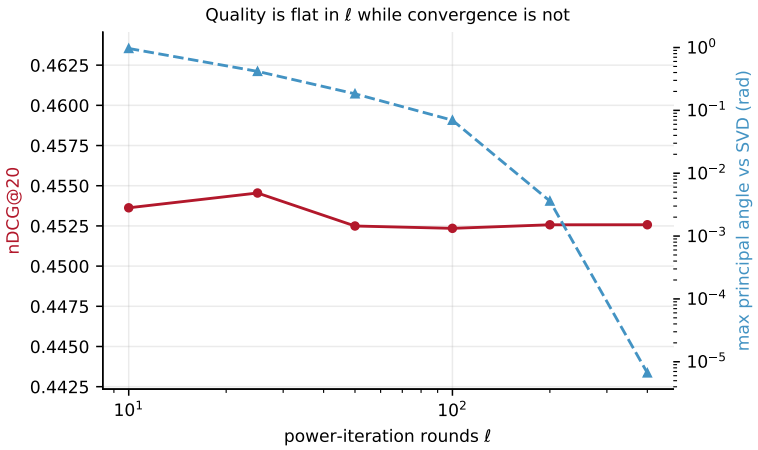


Figure 3: The cleartext oracle at ℓ from 10 to 400. nDCG@20 spans 0.002 while the subspace angle spans five orders of magnitude. Data: `bench/results/oracle_ell_sweep.jsonl`.

The exception at $\ell = 1$. With a single power iteration the no-leak path scores 0.1651 against 0.3729. The first normalisation acts on the raw random start vector, and a single iteration leaves nothing downstream to recover from whatever error it introduces. We have not isolated the mechanism further. From $\ell = 2$ the gap closes. Nobody would choose $\ell = 1$, but the cliff is sharp and we report it.

Quality is flat in ℓ. The oracle's own sweep settles how many iterations are worth paying for (Figure 3). Across a fortyfold increase in ℓ the subspace angle to the true singular subspace falls by five orders of magnitude, from 0.97 radians to 6.7×10^{-6}, while nDCG@20 stays between 0.4523 and 0.4545. Ranking needs a subspace good enough to order the scores stably, not the singular vectors themselves. The slow convergence has a visible cause: the sixteenth and seventeenth squared singular values differ by only 3%. Since rounds grow linearly in ℓ (Section 6.5), this flatness is what makes $\ell = 10$ the right operating point.

6.3 The cost of each half, isolated

The central comparison runs five arms in one process, interleaved repetition by repetition so that thermal drift cannot line up with arm identity (Table 6 and Figure 4). B1 trains in the clear and fetches in the clear. B2 swaps in DPF-PIR. B3 swaps in private training instead. FULL is OblivRec. B5 trains in the clear and downloads the whole catalogue. Training is amortised over the 943 users one training run serves.

Composing costs exactly the sum of the halves. FULL = B2 + B3 − B1 holds to the byte. Over a whole training run plus one session it is 64,760,124 bytes, and the benchmark asserts the identity before it writes a single row, so a regression that broke additivity could not produce a figure. On `wan_a`, private delivery adds 2.5 ms per session and private training adds 96 ms, and quality moves from 0.454 to 0.451.

The result that goes against us. On `wan_a`, B5 is 4.6 times *faster* than B2, although it sends 45 times as many bytes. Delivery as built costs one round trip per recommendation, ten in all, while

Table 6: Per-session cost of each arm, with training amortised over 943 users. Bytes and rounds are exact. Times are channel-level emulation. Data: `bench/results/bench_compose.jsonl`.

Arm	training, delivery	bytes	rounds	`local` (ms)	`wan_a` (ms)	nDCG@20
B1	clear, clear	2,600	10	1.19	301.4	0.454
B2	clear, DPF-PIR	9,660	10	3.11	303.9	0.454
B3	private, clear	71,264	13	2.02	397.7	0.451
FULL	private, DPF-PIR	78,324	13	3.96	400.2	0.451
B5	clear, full download	430,592	1	1.16	65.6	0.454

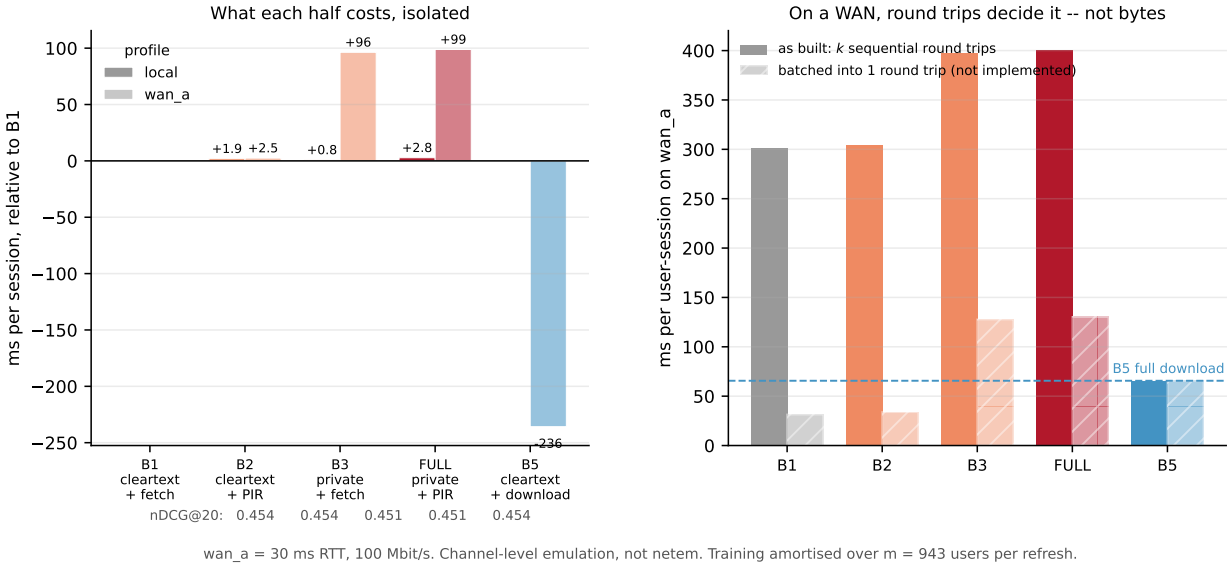


Figure 4: Left: what each arm adds over B1, per session, on `local` and `wan_a`. Right: the `wan_a` session time as built, against the modelled time with the k queries batched into one round trip, which is not implemented. Data: `bench/results/bench_compose.jsonl`.

B5 pays one. At 30 ms per trip, round trips decide the outcome and bytes barely matter. Sending all ten key pairs in one round trip reverses it: the model puts batched B2 at 33.9 ms against B5's 65.6 ms. We did not implement batching, and the benchmark marks those rows as not implemented. The honest summary is that private delivery as built is cheap in bytes and expensive in round trips, and the fix is known and unbuilt.

6.4 The private read against its alternatives

Figure 5 compares one private read with a cleartext lookup (B1) and with downloading the whole catalogue (B5), the first alternative a reader proposes because 430 KB does not sound large. DPF-PIR sends 483 bytes to each server, so it moves 446 times fewer bytes than B5 counting both servers. It is also the slowest of the three in server CPU, about five times slower than B5's local copy in both committed runs (5.3 and 5.6), and four orders of magnitude slower than a cleartext lookup. Its advantage is entirely in communication.

Setting the extra server time against the saved bytes gives the link speed below which the private read wins. That crossover is derived from absolute times, so it moves with the machine's thermal state: 8.0 Gbit/s in one committed run and 22.9 Gbit/s in the run the figure shows. Either way it is

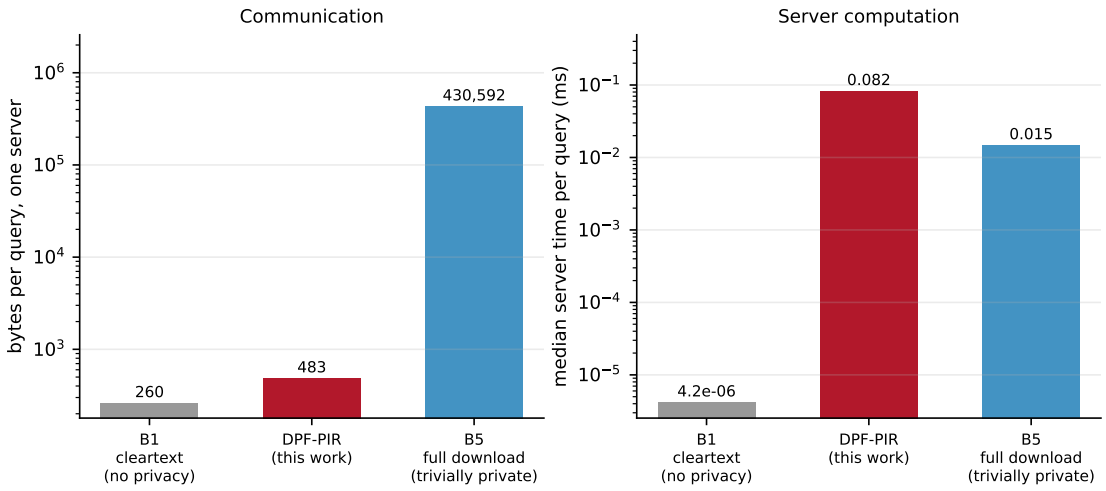


Figure 5: One read at the MovieLens-100K operating point: 1,682 records of 256 B in an 11-bit domain. Bytes are per server. Both panels must be read together, since the left alone would be advocacy. Data: `bench/results/bench_baseline.jsonl`.

far above any wide-area link, and we would rather state the crossover than imply there is none.

6.5 Scaling, and Theorem 4.2 measured

The sweep varies one parameter at a time around the baseline, five repetitions each, on all four profiles (Figure 6). It is not the Cartesian product, so interactions between parameters are not measured.

Rounds. Rounds are linear in d (1,310, 2,686 and 5,438 at $d = 8, 16, 32$) and in ℓ (436 at $\ell = 1$, 6,436 at $\ell = 25$), and flat in m: 2,686 at 235, 471 and 943 users.

Bytes. This is Nudge's Theorem 4.2 measured rather than quoted. Quadrupling the input matrix, from 235 to 943 users, moves traffic from 62.03 to 64.75 MB, an increase of 4.4%. The increase is exactly linear at 3,840 bytes per added user, which is the one term in the cost that depends on m: each of the $d\ell = 160$ iterations computes $\mathbf{Uv}$, an m-vector, and resharing it costs three parties one 8-byte element per entry, so $160 \times 3 \times 8 = 3{,}840$ bytes per user. Communication tracks the largest intermediate vector, not the input matrix.

Training is round-bound. On `wan_a` one training run takes 85.8 s, of which 80.6 s is round-trip latency and 5.2 s is transfer. Anything that removes rounds matters far more than anything that removes bytes, which is why the next subsection counts rounds.

6.6 Where the rounds go

Table 7 attributes every round and byte of one training run to the stage that spent it, in the three configurations. The attribution is a closed form derived from each protocol's structure, then validated against a real run: the residual is zero rounds and zero bytes in all three configurations, and the benchmark exits with an error if it is not. So these are shares of a complete budget, not a

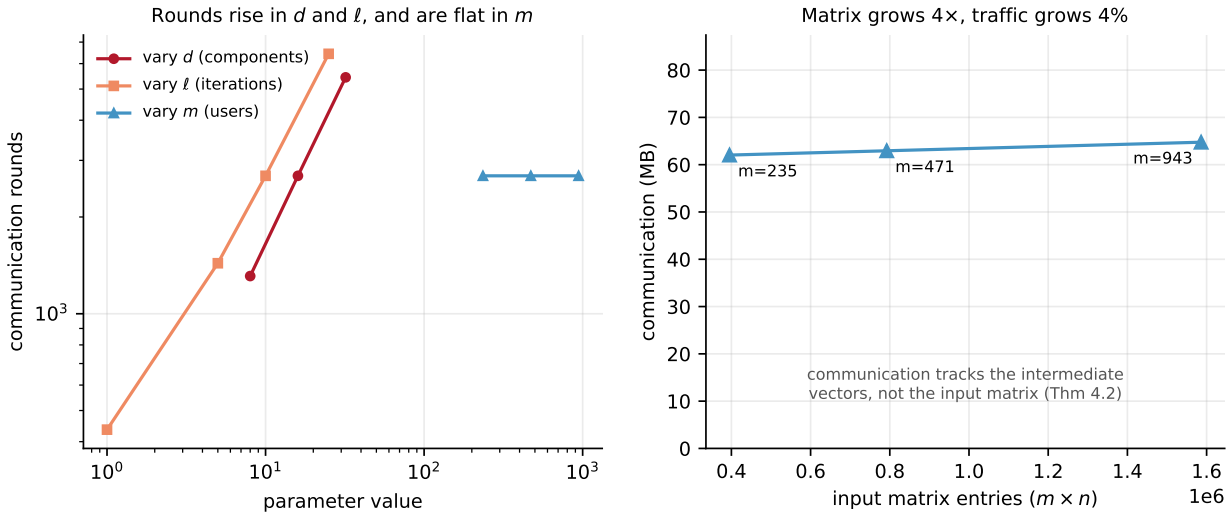


Figure 6: Left: rounds against d, ℓ and m. Right: traffic against the size of the input matrix. The matrix grows fourfold and traffic by 4.4%. Data: `bench/results/bench_sweep.jsonl`.

Table 7: Rounds per training run by stage, with each stage's share of rounds and of bytes. "Open **B**" is the sixteen row openings. Data: `bench/results/bench_stages.jsonl`.

	reveal, $b = 64$		reveal, $b = 128$		no leak, $b = 128$	
Stage	rounds	% rounds / bytes	rounds	% rounds / bytes	rounds	% rounds / bytes
matrix–vector	320	11.9 / 15.6	320	14.6 / 15.6	320	2.8 / 14.0
truncation	1,998	**74.4** / **83.4**	1,503	**68.6** / **83.4**	8,367	**72.6** / **75.1**
normalisation	352	13.1 / 0.0	352	16.1 / 0.0	2,464	21.4 / 9.9
FSS gate	0	0 / 0	0	0 / 0	352	3.1 / 0.0
open **B**	16	0.6 / 1.0	16	0.7 / 1.0	16	0.1 / 0.9
total	2,686		2,191		11,519	

sample. A zero residual alone would not catch rounds attributed to the wrong stage, so a second, independent count checks that too. A wrapper around the normaliser meters every round spent inside it, and the model must predict that count exactly, including the truncations the normaliser makes internally. It does in all three configurations: 880 rounds for each revealing path and 10,208 for the path that reveals nothing.

Truncation dominates, in every configuration. It is 68.6 to 74.4% of rounds and 75.1 to 83.4% of bytes. The FSS comparison gate, in the one configuration that has it, is 3.1% of rounds. The shared matrix–vector products cost 320 rounds, one per product, and between 2.8 and 14.6% of rounds. Anyone optimising this system should start with truncation, whose three rounds recur after every rescaling, every orthogonalisation and every Newton step. The gate, which looks like the exotic part, is not the problem.

CPU ranks the stages differently. Figure 7 also times each operation on its own. The shared matrix–vector product over the full rating matrix is the most expensive operation in CPU by a wide margin, while being cheap in rounds. On a wide-area link the round count governs, and locally the

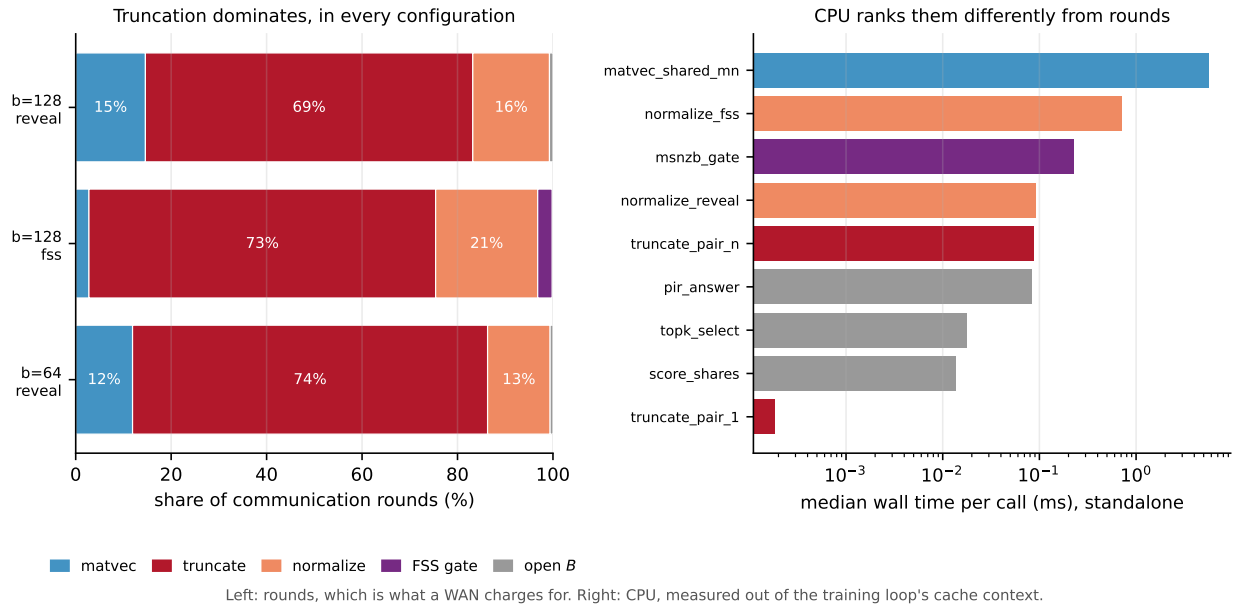


Figure 7: Left: each stage's share of the rounds in one training run, per configuration. Right: median CPU time per call, each operation timed on its own. Data: `bench/results/bench_stages.jsonl`.

CPU time does, so which stage matters depends on where the servers are.

6.7 How wide the ring must be

Whether a 64-bit ring suffices at MovieLens scale, or NUDGE's 128 bits are needed, was an open question in our design. It has two answers, for two different reasons, and the tighter one wins.

Arithmetic headroom. The truncation schedule is derived from public parameters, and deriving it at larger scales answers the question without running them. At $t = 20$ and $b = 64$, the deferred schedule does not fit at any of the three scales we evaluated, MovieLens-100K, MovieLens-1M and a Netflix-sized matrix, so training truncates immediately, with 21, 18 and 11 bits to spare. At $t = 24$ and Netflix size there is no safe schedule at all. At $b = 128$ the deferred schedule fits everywhere with at least 55 spare bits, and at MovieLens-100K it needs fewer rounds (2,191 against 2,686) for twice the bytes (129.3 MB against 64.75 MB).

The mask. The spec-faithful normaliser's gate needs an 81-bit domain to hide its input (Section 5.5). So a 64-bit ring is enough arithmetically at MovieLens-1M, and not enough at any scale for a normaliser that reveals nothing. That second constraint is not arithmetic, and it is the one that decides the ring width.

6.8 On the wire

An argument that message sizes do not depend on the fetched index, and a measurement that they do not, are different claims. We recorded every frame at the channel boundary while a probe issued 1,800 queries across three record indices, 0, 777 and 1234, generating fresh keys for every query. Every query frame is 228 bytes and every reply is 265 bytes, whatever the index.

Table 8: Telling fetched indices apart from the wire bytes alone, with 480 held-out queries per pair and the same pipeline run on shuffled labels as a control. Data: `bench/results/distinguisher_result.jsonl`.

Adversary	accuracy	95% interval
index 0 against 1234	0.490	0.445 to 0.534
index 0 against 777	0.494	0.449 to 0.538
index 1234 against 777	0.500	0.455 to 0.545
shuffled labels	0.483	0.439 to 0.528

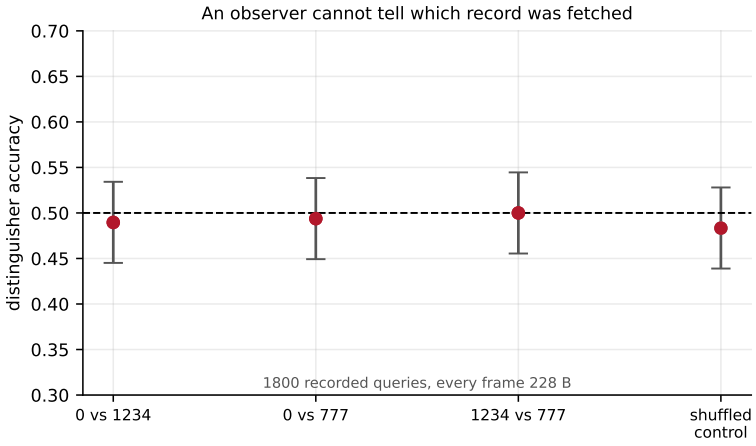


Figure 8: The distinguisher at chance. What matters is that 0.5 lies inside every interval, not that the points sit near it. Data: `bench/results/distinguisher_result.jsonl`.

We then trained a logistic-regression classifier on the raw frame bytes to tell each pair of indices apart and scored it on 480 held-out queries per pair (Table 8 and Figure 8). Chance lies inside every 95% interval. The same pipeline run on deliberately shuffled labels agrees with the real runs, which shows the harness measures what it claims to.

This is not a proof. It says that one adversary, with one feature representation, did not beat chance on this data. What it rules out is the failure that actually occurs in practice, a length or byte pattern that correlates with the query.

6.9 Server-side timing

Frame sizes are fixed, but a server whose running time depended on the index would leak it to anyone who can time the reply. We timed the PIR answer routine 200 times at each of eight indices spread across the domain, interleaved rather than in blocks so that thermal drift could not line up with the index (Figure 9). The mean call takes 87 μs. The spread between the eight per-index means is 1.7 μs, smaller than the median standard deviation of 6.0 μs within a single index, and a permutation test on shuffled index labels puts the observed spread at $p = 0.60$, in the middle of the null distribution.

This is a bound, not a proof of constant time, because no finite sample shows a difference is exactly zero. What it shows is that any index-dependent signal is below 2.0% of the mean call at this sample size. That is consistent with the construction, which expands every node and takes

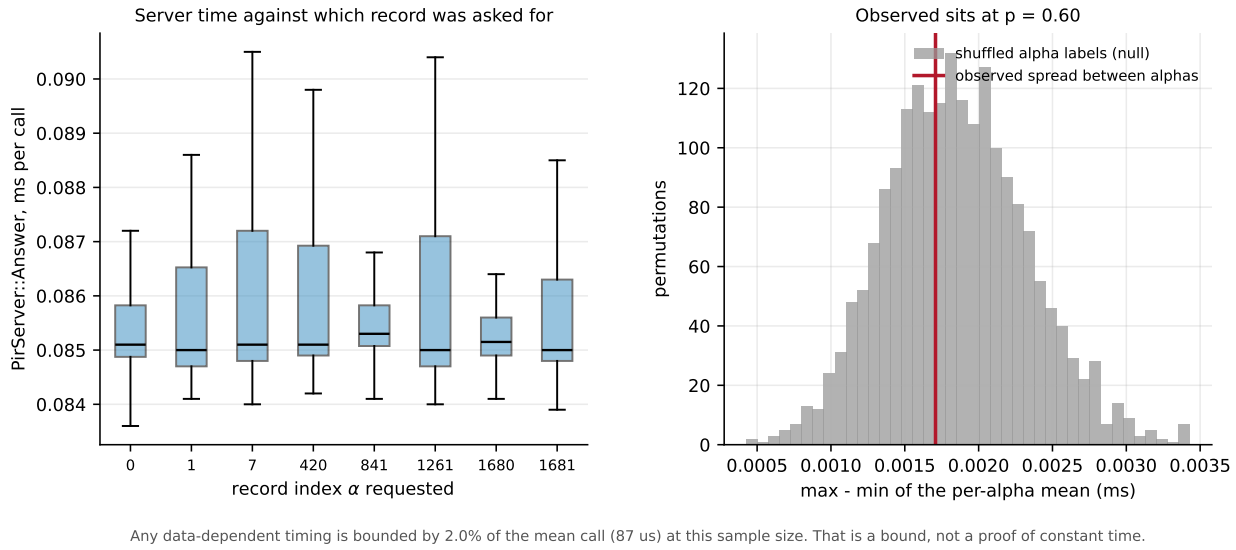


Figure 9: Left: server time per call at eight record indices. Right: the spread between index means, against its distribution under shuffled labels. Data: `bench/results/bench_timing.jsonl`.

an unconditional inner product over every record. Timing on the wire itself is dominated by the operating system's scheduler and network stack, and we did not analyse it.

7 Leakage of the composition

Section 6 priced the private read. This section asks what it buys: what an observer learns when the fetch is *not* private, whether anything cheaper than a private read closes the gap, and what a malicious client can do to the composed system. Neither base paper can answer these, because neither implements both halves.

7.1 Why a fetch is a projection

B is published, so every observer holds a complete latent-factor model of the catalogue. The items recommended to user i are the ones whose columns of **B** score highest against the private embedding $\mathbf{a}^{(i)}$. An observer who sees *which* items the user fetched therefore learns k vectors that all point roughly where $\mathbf{a}^{(i)}$ points. Each fetch is a constraint on the private embedding, expressed in a basis the observer already has. The question is how quickly those constraints give it away.

7.2 The attack, and the control that makes it a result

The adversary is given **B** and the indices of the first j records a user fetched, and nothing else: no scores, no ratings, no part of **A**. It estimates $\hat{\mathbf{a}}$ as the mean of the fetched items' unit-normalised embeddings. We report the cosine between $\hat{\mathbf{a}}$ and the true $\mathbf{a}^{(i)}$, and the share of the user's *next* twenty recommendations that the ranking induced by $\hat{\mathbf{a}}$ predicts, excluding the j items already seen. All 943 users are attacked, against the same model the demo serves.

Table 9: Recovering a user's private embedding from observed fetches alone, averaged over all 943 users. Data: `bench/results/leakage_attack.jsonl`.

Fetches j	cos($\hat{\mathbf{a}}$, $\mathbf{a}$)	popularity control	margin	next-20 overlap
1	0.55	0.26	0.28	39%
2	0.66	0.26	0.40	47%
5	0.77	0.39	0.38	53%
10	**0.84**	0.44	0.40	**56%**
20	0.89	0.47	0.42	55%
50	0.91	0.53	0.39	46%

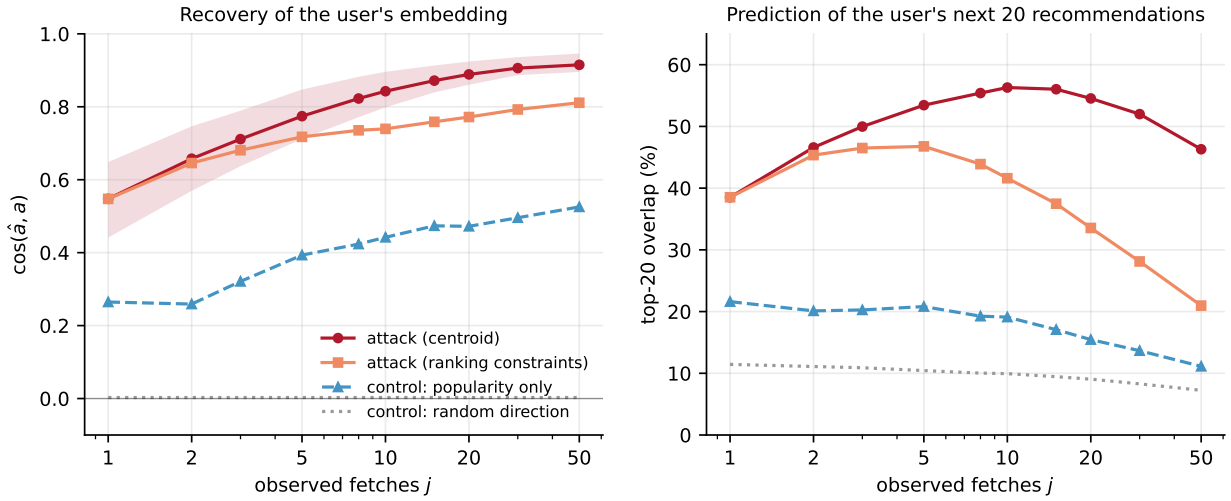


Figure 10: Left: cosine between the estimate and the true embedding, with the interquartile band of the attack. Right: the share of the user's next twenty recommendations the adversary names. The gap between the attack and the popularity control is the part that is about the individual. The second attack curve is a stronger-looking estimator that uses the ranking constraints, and it does worse. Data: `bench/results/leakage_attack.jsonl`.

The control. Popular items sit near the top of almost everyone's ranking, so an estimator can look strong while having learned only what is public. The control therefore runs the identical estimator on the j globally most popular items, ignoring the user entirely, and the attack may claim only what it achieves above that line. A random direction is also reported, to show where the floor of pure guessing lies.

One observed fetch gives a cosine of 0.55, twice the popularity control. Ten give 0.84, enough to name 56% of the user's next twenty recommendations (Table 9, Figure 10). The margin over the control is 0.28 at one fetch and between 0.38 and 0.42 from two fetches on. The random direction averages 0.003.

We raise three caveats ourselves. First, one user's cosine is not evidence. The random control's interquartile range runs from −0.17 to 0.18, which is what a random direction in sixteen dimensions gives, so a single user at 0.2 is inside the noise of guessing. Only population means and the margin over the control carry a claim. Second, the overlap column rises and then falls, peaking near ten fetches. That is the metric, not the attack: the observed items are removed from the candidates, so the easy head of the ranking is progressively taken away, while the cosine keeps rising. Third, a

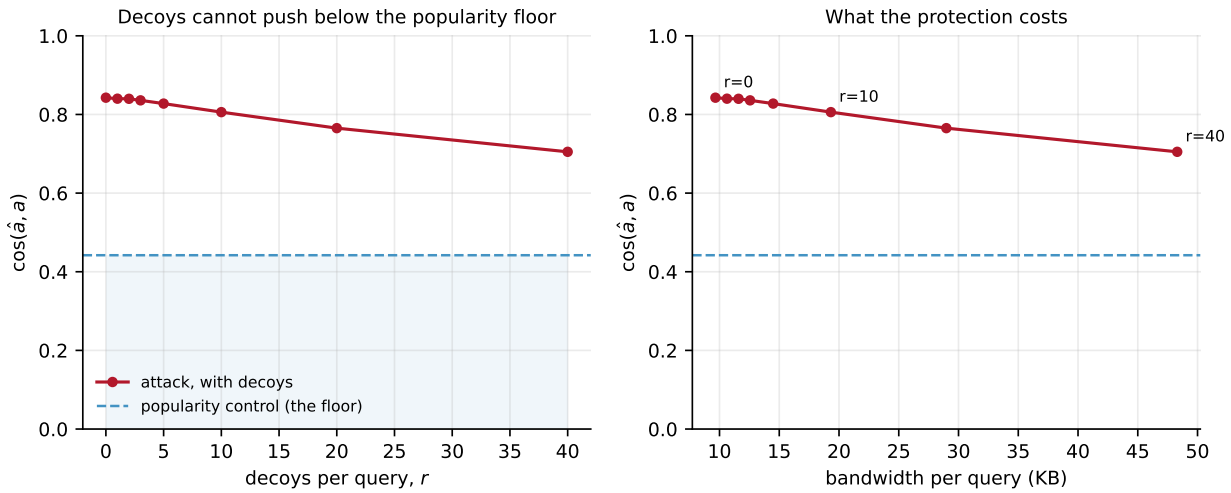


Figure 11: The attack at ten real fetches plus r popularity-weighted decoys, against decoys per query (left) and against traffic per query (right). The dashed line is the popularity control. Data: `bench/results/leakage_attack.jsonl`.

second estimator that uses the fact that every fetched item outranks every unfetched one, strictly more information than the mean uses, does *worse* from two fetches on, 0.81 against 0.91 at fifty. That sharpens the conclusion. The cheap, obvious attack is already enough, and an adversary needs no sophistication.

7.3 Defence one: decoys

The obvious cheap alternative to a private read is to fetch the k real recommendations together with r decoys. The decoys must be drawn in proportion to popularity, because real recommendations are popular and uniformly drawn decoys would come from the long tail, where an observer could discard them by inspection and attack what remained.

It does not work (Figure 11). Forty decoys, five times the real traffic, lower the attack's cosine only from 0.843 to 0.705, against a popularity control at 0.442. The reason is the same fact that made the control necessary. Popularity-weighted decoys point along the popularity direction, which is exactly what the control already recovers, so decoys drag the estimate *towards* the control and no further. The defence has a floor built into it, and the floor is where an adversary who watched nobody already stands.

7.4 Defence two: Nudge's differential privacy

Nudge's Section 9 extends the system with differential privacy. The servers normalise each user's row of $\mathbf{U}$, share a symmetric Gaussian matrix E once through their PRF keys, and add $E\mathbf{v}$ at every step of power iteration, so the iteration runs on $\mathbf{U}^\top\mathbf{U} + E$ at no extra communication. The paper does not state the noise scale. We take it from the Gaussian mechanism of Dwork, Talwar, Thakurta and Zhang that it cites [DTTZ14]: with every row of unit norm, one user changes $\mathbf{U}^\top\mathbf{U}$ by at most one in Frobenius norm, and $\sigma = \sqrt{2\ln(1.25/\delta)}/\varepsilon$ gives (ε, δ)-differential privacy [DR14]. We use $\delta = 2^{-40}$, as Nudge does.

We deviate from the paper in one place, deliberately. Nudge normalises row i by dividing by

Table 10: NUDGE's mechanism on MovieLens-100K at $\delta = 2^{-40}$, with the attack at ten fetches. Data: `bench/results/dp_study.jsonl`.

	nDCG@20	attack cosine	margin over control
no noise (rows normalised)	0.4255	0.8231	0.4324
$\varepsilon = 8$	0.1361	0.9167	0.5044
$\varepsilon = 1$	**0.1239**	**0.9212**	**0.4898**

the user's number of ratings c_i. With ratings up to five, that bounds the row's norm by $5/\sqrt{c_i}$, which exceeds one whenever $c_i < 25$, and in the training split we use, 200 of the 943 users have fewer than 25 ratings. For them the privacy bound would not hold, so we normalise each row to unit ℓ_2 norm instead. The count-normalised arm was run too and is in the data. It behaves the same way.

Both axes move the wrong way. At $\varepsilon = 1$, quality falls by 71% and the attack gets *stronger* (Table 10). Even at $\varepsilon = 8$ quality falls to 0.136. The second result is not a paradox. Differential privacy here protects the *training data*: it bounds what **B** reveals about any user's ratings. The attack recovers *this* user's embedding using **B** as a basis, and the user and the adversary work against the same published, noisy matrix, so the noise does not disturb the attack's geometry. What the noise does destroy is the shared popularity structure the control exploits, which widens the attack's margin.

Why we lose 71% where NUDGE loses 17%. NUDGE reports nDCG@20 falling from 0.29 to 0.24 on Netflix [HDCB26]. The mechanism is the same, and the dataset is not. The noise matrix's spectral norm grows as $\sigma\sqrt{n}$ while the signal, the spectral norm of $\mathbf{U}^\top\mathbf{U}$, grows with the number of users m. The usable regime is set by $m/\sqrt{n}$, which is 157 times larger for Netflix, with 480,189 users and 17,770 items, than for MovieLens-100K. Measured directly, $\|\mathbf{U}^\top\mathbf{U}\|_2 = 144.6$ against $\|E\|_2 \approx 613$ at $\varepsilon = 1$, so the noise dominates the signal by a factor of 4.2 (Figure 12). This is a statement about our dataset's scale, not a contradiction of the paper.

Neither cheaper defence reaches the popularity control. Decoys cost bandwidth and stay far above it, and differential privacy costs most of the model's quality and moves the attack further away from it. That is the argument for making the read itself private, and it is the gap NUDGE leaves to "other means".

7.5 The malicious client

The last adversary class the composition creates is a client who does not follow the protocol. For retrieval alone, a malformed query only corrupts that client's own answer. The harvest changes that, because a query now also writes into the next model.

The hole. An honest client generates its keys with $\beta = 1$, so the two expansions differ by exactly the indicator of the fetched item. Nothing stopped a client from generating keys with any other β. Retrieval still works, since the record comes back scaled by β and the client divides it out, and the servers add β, not one, into the consumption counts. Every existing check passes: the keys parse, the domain matches, the expansion has the right length. Our test shows one query at $\beta = 10^6$ placing 1,000,000 into the next round's training input.

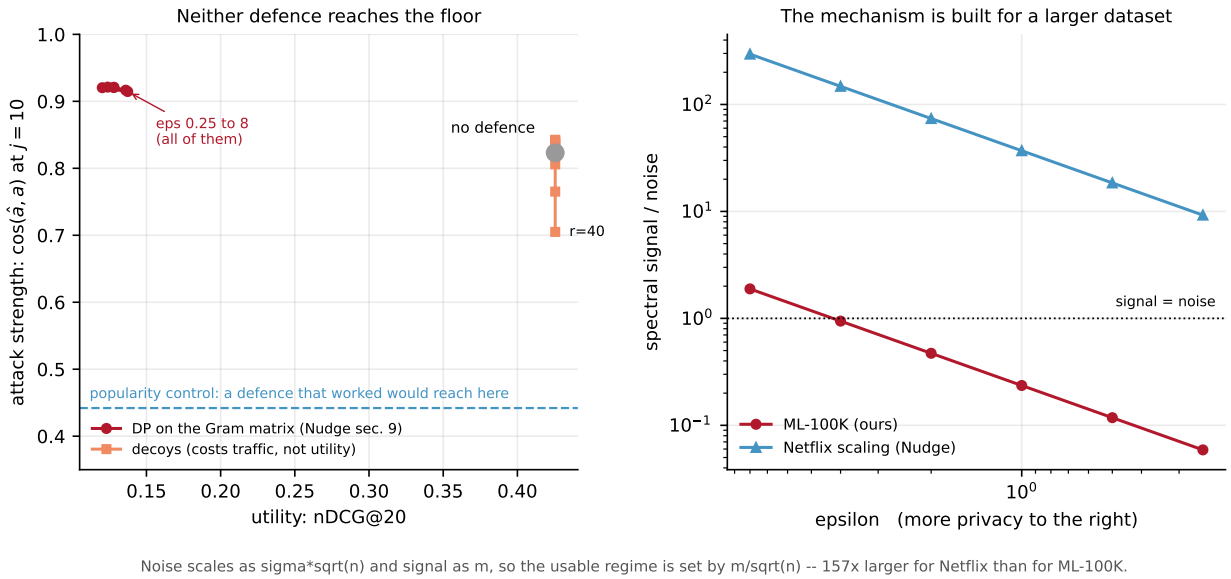


Figure 12: Left: both cheaper defences in one plane, attack strength against recommendation quality. A defence that worked would reach the popularity control. Right: the spectral signal-to-noise ratio against ε for our dataset and for Netflix's dimensions. Data: `bench/results/dp_study.jsonl` and `bench/results/leakage_attack.jsonl`.

What it buys. Less than we first assumed, and for an instructive reason (Figure 13, left). A single attacker at weight 10^6 lowers honest users' nDCG@20 by between 0.7 and 1.0%, depending on its targets, and the items it promotes do not rise. Their mean rank worsens, from 901 to 973 of 1,682. Power iteration normalises at every step, so beyond some point extra weight only fixes a direction, and one attacker captures one of the $d = 16$ components while the other fifteen still carry the honest signal. Colluders with distinct targets are another matter. Eight cost 4.1%, and sixteen take nDCG@20 from 0.4536 to 0.0021. We ran only powers of two, so the data place the collapse somewhere between eight and sixteen colluders, and it is consistent with the attackers capturing all d components. This is destruction of the model, not promotion of items.

The check. Summation is linear, so each PIR server can sum its own expansion locally, and the two sums differ by exactly β, whatever the index. The servers open that one scalar and require it to equal one: one round and two ring elements per query. Opening it reveals nothing about the index, because β is meant to be the public constant one and each server's own sum is pseudorandom. Our test confirms the check accepts $\beta = 1$ and rejects $\beta = 10^6$ at that cost.

What the check does not close. It bounds the total weight of a query, not how the weight is distributed. A client that forges the expansion directly, rather than calling the key generator, can place +2 at one item and −1 at another, which sums to one and passes. Our test constructs exactly that forgery and shows it passes. A forgery of the same shape with large opposite weights would give colluders the same power as before, and we have not measured it. Closing it requires proving that a key encodes a genuine point function, which is what the audit of Sabre [VSH22] does, and we did not build it.

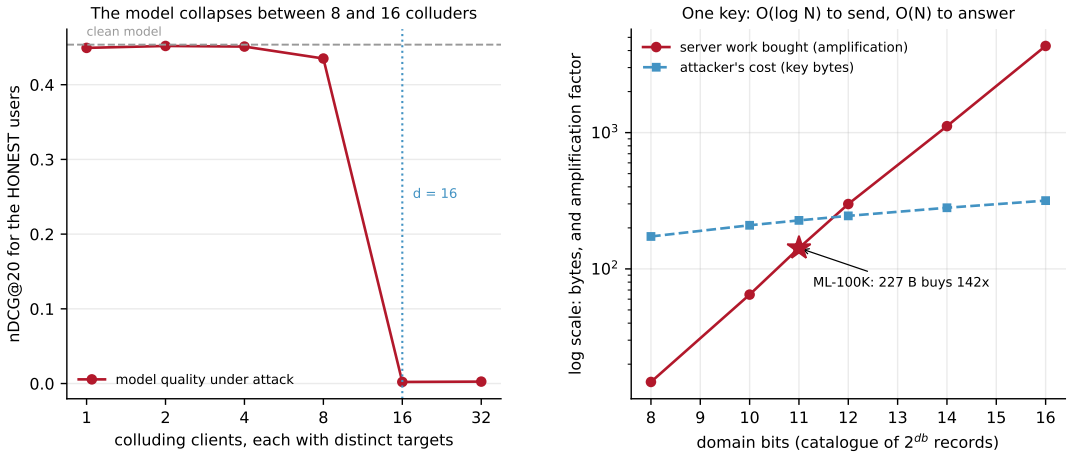


Figure 13: Left: honest users' nDCG@20 against the number of colluding clients, each voting at weight 10^6 for distinct targets, before the weight check. Only powers of two were run. Right: server work bought by one key, and the key's size, against the domain. The star is our operating point. Data: `bench/results/poison_study.jsonl` and `bench/results/bench_dos.jsonl`.

Denial of service. Answering any query means expanding the key over the whole domain, while the key is only logarithmic in the domain. So a query buys server work out of proportion to its size (Figure 13, right). At our 11-bit domain, a 227-byte key buys 142 times its own cost in server time in the run the figure shows, and 121 times in an earlier committed run. At 16 bits the same run measures 4,323 times. The amplification grows with the catalogue, the wrong direction for a defender. Malformed keys are cheap to reject, since parsing validates them before any expansion, so the expensive attack is the one that looks legitimate. The linear work is not an inefficiency. It is the privacy property, since any shortcut that depended on the index would reveal it. This is the clearest example in the project of privacy paid for in availability, and the same unbuilt audit is what would address it.

8 Related work

Our literature survey, submitted as the project's first milestone, covers the field in full. This section places OblivRec against the work it builds on and the work closest to it.

Pirsona. Pirsona [VBH21] is the system closest in spirit, and the only one we found that spans collection, training and delivery. Its authors describe private information retrieval and collaborative filtering as "seemingly antithetical primitives": PIR exists to hide which item a user fetches, and collaborative filtering works by relating users through exactly that. Its resolution is the harvest we reuse. Delivery uses the multi-server PIR of Hafiz and Henry [HH19], the servers extract secret-shared consumption from the query traffic itself, and training runs a four-party Boolean matrix factorisation over it. OblivRec keeps the harvest and the PIR layer and replaces the training core with Nudge's, one fewer non-colluding party and an arithmetic rather than Boolean computation.

Nudge. Nudge [HDCB26] is our training core. Users secret-share ratings, three servers factorise by power iteration, and the servers compute each user's scores in shared form. Its non-goal is our starting point: it "relies on other means (e.g., Apple's private relay, Tor, or cryptographic private information retrieval) to let users fetch data items in a private way". We add three things it does not report: a private fetch composed with its training, a measurement of what leaks when the fetch is not private, and a measurement of its own differential-privacy extension against that leak.

Private training more broadly. The first practical private matrix factorisation, that of Nikolaenko et al. [NIW+13], evaluates gradient descent inside a garbled circuit across two servers.[1] A garbled circuit's communication scales with the circuit, and so with the whole rating matrix, and later work moved to secret sharing. General frameworks such as MP-SPDZ [Kel20] and the ABY family [DSZ15, PSSY21] could run the same factorisation, and a generic MPC baseline was part of our original evaluation plan. We did not build it, for reasons Section 9 gives. The fixed-point costs that dominate our measurements are a known theme: SecureML [MZ17] and Escudero et al. [EGK+20] both treat truncation as a first-class cost.

Function secret sharing and PIR. The DPF is that of Boyle, Gilboa and Ishai [GI14, BGI15, BGI16], and the comparison gate follows the FSS-based secure computation of Boyle et al. [BCG+21], which evaluates comparison gates in a constant number of rounds by masking the input. Two-server PIR from a DPF is a special case of Hafiz and Henry's family [HH19]. Single-server PIR such as SimplePIR [HHC+23] removes the non-collusion assumption at the price of a large client-side hint, and Floram [DS17] and Duoram [VHG23] build distributed oblivious memory from the same DPF machinery we use. Sabre's audit [VSH22], which proves that a DPF key encodes a genuine point function, is the mechanism our malicious-client analysis names as the missing piece.

Private retrieval without a recommender. A separate line of work retrieves privately over embeddings, from secure nearest-neighbour search [CCD+20] to private web search [HDCZ23] and more recent sublinear schemes [ZSF25, ZPZP25, LHZ+25, ABG+24]. These take a corpus and its embeddings as given and are not connected to a privately trained model. Pirsona predates almost all of them, which is part of why the composition we build had not been attempted with a modern training core.

What leaks through the output. Calandrino et al. [CKN+11] address the privacy risks of collaborative filtering itself.[2] Our reconstruction attack concerns a narrower channel. It needs no access to the recommender's output, only to which items a user fetched, together with the public **B** that Nudge's design releases.

9 Limitations

We collect here everything a reader should discount the results by. Most of it has been said at the point of use. It is repeated because a limitation stated once, in the middle of a paragraph about something else, is easy to miss.

[1] We could not obtain this paper. The characterisation follows Nudge's description of it, as our survey's did.
[2] We could not obtain this paper either, and cite it only for the claim its title makes.

Semi-honest, with at most one corrupted server. Every guarantee assumes the servers follow the protocol. A deviating server can corrupt the model and the answers, and nothing detects it. **Two colluding servers learn everything**: any two hold all three shares of every value, and the two PIR servers together reconstruct every fetched index. This is the assumption the whole system rests on, and it is NUDGE's own.

A sketch, not a proof. Section 2.5 walks every message and names a simulator, but there is no reduction and no hybrid argument. The revealing normaliser is secure only against the weaker $\mathcal{F}_{\text{train}}^{\text{leaky}}$, and every $b = 64$ quality number comes from it. What repeated training rounds reveal about a user whose consumption feeds them is open.

Scale. Every measurement is on MovieLens-100K. We derived the truncation schedule at MovieLens-1M and Netflix scale, but did not run private training there. A dense shared 6,040×3,706 matrix and its transpose, for three parties in one process, need about 2.1 GB, more than the development machine could spare. The ring-width conclusions at larger scale are therefore analytic.

Emulated networks, and one machine. Wide-area results come from channel-level emulation on a single laptop, not from servers in separate locations or from kernel network emulation, neither of which the machine supports. Training's wide-area times are computed from exact round and byte counts, which is faithful for a round-bound protocol but ignores congestion and jitter.

Absolute times are unreliable. Thermal throttling moves absolute timings by up to a factor of four between runs. We quote ratios, and where a derived number moved between committed runs we give both values: the PIR crossover link speed (8.0 and 22.9 Gbit/s) and the denial-of-service amplification (121 and 142 times). Anyone needing absolute numbers should rerun on a machine that holds its clock.

One factor at a time. The sweep varies each parameter around one baseline. Interactions between parameters, for instance between d and b, are not measured.

Delivery is not batched. Each recommendation costs a round trip, so on a wide-area link the full catalogue download is 4.6 times faster than our private read. Batching the k queries into one trip is straightforward and would reverse the result by our model, but it is not built.

The harvest runs as a library path, not in the deployed loop. The harvest, its conversion and the weight check are implemented and tested, but the networked servers do not yet accumulate consumption across sessions or retrain from it. Training in every experiment reads MovieLens ratings from disk. The loop is closed in code and in tests, not in a long-running deployment.

Malicious clients are only partly handled. The weight check stops a client from inflating its vote. It does not stop a forged key from redistributing weight, and it does nothing about the denial of service that full-domain evaluation invites. Both need an audit in the style of SABRE, which we did not build. The poisoning study places the collapse between eight and sixteen colluders because only powers of two were run.

Leakage we do not analyse. Timing on the wire, as opposed to server compute time, is dominated by the operating system and was not studied. Query counts and timing are leaked by design. The number of ratings each user contributes is leaked unless every user submits the same number. And hiding the fetch does nothing about what the recommendation reveals to anyone who later observes the user act on it.

The server download of scores. Because the user selects the top k locally, each server sends the user a share of all n scores. That is cheap at 1,682 items and grows linearly with the catalogue, where an oblivious selector run by the servers would send only k indices at a large cost in rounds.

No generic-MPC baseline. The evaluation plan included running the same factorisation in a general MPC framework. That needs Linux or Docker, which the development machine lacks, so the comparison with generic MPC is absent.

Evaluation choices. One dataset split, one seed for the attack, and $d = 16$ throughout the leakage study. Our nDCG uses binary relevance at a rating of four or more, so absolute values are comparable within this report and not with Nudge's numbers.

10 Conclusion

Nudge trains a recommender privately and stops at the scores. Pirsona delivers privately with a training core the field has moved past. OblivRec composes the two and measures the result, and three findings stand out.

First, the composition is cheap in the way that matters to its designers and expensive in a way they did not measure. It costs exactly the sum of its halves, private training costs 96 ms per user-session on a wide-area link when amortised over 943 users, and the normaliser that reveals nothing loses only a few thousandths of nDCG@20. But the rounds are dominated by truncation, not by the comparison gate that looks like the hard part, and delivery as built pays a round trip per recommendation, which lets a full catalogue download beat it on a wide-area link.

Second, the private read is necessary, and the cheaper alternatives do not replace it. Because the item embeddings are public by design, ten fetches seen in the clear recover a user's taste to a cosine of 0.84. Decoys at five times the traffic leave the attack far above what popularity alone gives, and Nudge's own differential privacy strengthens the attack while removing most of the model's quality at our scale, for a reason that the ratio $m/\sqrt{n}$ explains.

Third, composing creates a surface neither paper had. Once delivery feeds training, a client's query is also a vote, and an unchecked vote of weight 10^6 is available to anyone who calls the key generator directly. One round closes inflation. Redistribution and the denial of service that full-domain evaluation invites remain open, and both point to the same missing piece, an audit that a key encodes a genuine point function.

The most useful practice in the project was measuring quality alongside cost. A benchmark that counted only rounds and bytes would have reported that revealing nothing costs 24% of recommendation quality, when the real cost was a silently clamping comparison range, and the true figure is two thousandths.

Next steps. In order of value, the first is to batch the k PIR queries into one round trip, which our model says reverses the wide-area result. The second is to drive the harvest from the networked

servers, so that the loop runs as a deployment and not only in tests. The third is the key audit, and the fourth is private training at MovieLens-1M on a machine with the memory for it.

References

[ABG+24] Hilal Asi, Fabian Boemer, Nicholas Genise, Muhammad Haris Mughees, Tabitha Ogilvie, Rehan Rishi, Guy N. Rothblum, Kunal Talwar, Karl Tarbe, Ruiyu Zhu, and Marco Zuliani. Scalable private search with wally. *CoRR*, abs/2406.06761, 2024.

[AFL+16] Toshinori Araki, Jun Furukawa, Yehuda Lindell, Ariel Nof, and Kazuma Ohara. High-throughput semi-honest secure three-party computation with an honest majority. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, Vienna, Austria, October 24-28, 2016*, pages 805–817. ACM, 2016.

[BCG+21] Elette Boyle, Nishanth Chandran, Niv Gilboa, Divya Gupta, Yuval Ishai, Nishant Kumar, and Mayank Rathee. Function secret sharing for mixed-mode and fixed-point secure computation. In *Advances in Cryptology - EUROCRYPT 2021*, pages 871–900. Springer, 2021.

[BGI15] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing. In *Advances in Cryptology - EUROCRYPT 2015*, pages 337–367. Springer, 2015.

[BGI16] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing: Improvements and extensions. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, Vienna, Austria, October 24-28, 2016*, pages 1292–1303. ACM, 2016.

[CCD+20] Hao Chen, Ilaria Chillotti, Yihe Dong, Oxana Poburinnaya, Ilya P. Razenshteyn, and M. Sadegh Riazi. SANNS: scaling up secure approximate k-nearest neighbors search. In *29th USENIX Security Symposium, USENIX Security 2020*, pages 2111–2128. USENIX Association, 2020.

[CGKS95] Benny Chor, Oded Goldreich, Eyal Kushilevitz, and Madhu Sudan. Private information retrieval. In *36th Annual Symposium on Foundations of Computer Science, FOCS 1995*, pages 41–50. IEEE Computer Society, 1995.

[CKN+11] Joseph A. Calandrino, Ann Kilzer, Arvind Narayanan, Edward W. Felten, and Vitaly Shmatikov. "you might also like: " privacy risks of collaborative filtering. In *32nd IEEE Symposium on Security and Privacy, SP 2011*, pages 231–246. IEEE Computer Society, 2011.

[DR14] Cynthia Dwork and Aaron Roth. The algorithmic foundations of differential privacy. *Found. Trends Theor. Comput. Sci.*, 9(3-4):211–407, 2014.

[DS17] Jack Doerner and Abhi Shelat. Scaling ORAM for secure computation. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS 2017, Dallas, TX, USA, October 30 - November 03, 2017*, pages 523–535. ACM, 2017.

[DSZ15] Daniel Demmler, Thomas Schneider, and Michael Zohner. ABY - A framework for efficient mixed-protocol secure two-party computation. In *22nd Annual Network and Distributed System Security Symposium, NDSS 2015*. The Internet Society, 2015.

[DTTZ14] Cynthia Dwork, Kunal Talwar, Abhradeep Thakurta, and Li Zhang. Analyze gauss: Optimal bounds for privacy-preserving principal component analysis. In *Symposium on Theory of Computing, STOC 2014*, pages 11–20. ACM, 2014.

[EGK+20] Daniel Escudero, Satrajit Ghosh, Marcel Keller, Rahul Rachuri, and Peter Scholl. Improved primitives for MPC over mixed arithmetic-binary circuits. In *Advances in Cryptology - CRYPTO 2020*, pages 823–852. Springer, 2020.

[GI14] Niv Gilboa and Yuval Ishai. Distributed point functions and their applications. In *Advances in Cryptology - EUROCRYPT 2014*, pages 640–658. Springer, 2014.

[HDCB26] Alexandra Henzinger, Emma Dauterman, Henry Corrigan-Gibbs, and Dan Boneh. Nudge: A private recommendations engine. In *35th USENIX Security Symposium, USENIX Security 2026*. USENIX Association, 2026. Full version: IACR ePrint 2026/179.

[HDCZ23] Alexandra Henzinger, Emma Dauterman, Henry Corrigan-Gibbs, and Nickolai Zeldovich. Private web search with tiptoe. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023*, pages 396–416. ACM, 2023.

[HH19] Syed Mahbub Hafiz and Ryan Henry. A bit more than a bit is more than a bit better: Faster (essentially) optimal-rate many-server PIR. *Proc. Priv. Enhancing Technol.*, 2019(4):112–131, 2019.

[HHC+23] Alexandra Henzinger, Matthew M. Hong, Henry Corrigan-Gibbs, Sarah Meiklejohn, and Vinod Vaikuntanathan. One server for the price of two: Simple and fast single-server private information retrieval. In *32nd USENIX Security Symposium, USENIX Security 2023*, pages 3889–3905. USENIX Association, 2023.

[HK16] F. Maxwell Harper and Joseph A. Konstan. The movielens datasets: History and context. *ACM Trans. Interact. Intell. Syst.*, 5(4):19:1–19:19, 2016.

[HKV08] Yifan Hu, Yehuda Koren, and Chris Volinsky. Collaborative filtering for implicit feedback datasets. In *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM 2008*, pages 263–272. IEEE Computer Society, 2008.

[KBV09] Yehuda Koren, Robert M. Bell, and Chris Volinsky. Matrix factorization techniques for recommender systems. *Computer*, 42(8):30–37, 2009.

[Kel20] Marcel Keller. MP-SPDZ: A versatile framework for multi-party computation. In *CCS '20: 2020*, pages 1575–1590. ACM, 2020.

[LHZ+25] Jingyu Li, Zhicong Huang, Min Zhang, Cheng Hong, Jian Liu, Tao Wei, and Wenguang Chen. Panther: Private approximate nearest neighbor search in the single server setting. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security, CCS 2025, Taipei, Taiwan, October 13-17, 2025*, pages 365–379. ACM, 2025.

[MZ17] Payman Mohassel and Yupeng Zhang. Secureml: A system for scalable privacy-preserving machine learning. In *2017 IEEE Symposium on Security and Privacy, SP 2017, San Jose, CA, USA, May 22-26, 2017*, pages 19–38. IEEE Computer Society, 2017.

[NIW+13] Valeria Nikolaenko, Stratis Ioannidis, Udi Weinsberg, Marc Joye, Nina Taft, and Dan Boneh. Privacy-preserving matrix factorization. In *2013 ACM SIGSAC Conference on*

Computer and Communications Security, CCS'13, Berlin, Germany, November 4-8, 2013, pages 801–812. ACM, 2013.

[NS08] Arvind Narayanan and Vitaly Shmatikov. Robust de-anonymization of large sparse datasets. In *2008 IEEE Symposium on Security and Privacy (SP 2008), 18-21 May 2008, Oakland, California, USA*, pages 111–125. IEEE Computer Society, 2008.

[PSSY21] Arpita Patra, Thomas Schneider, Ajith Suresh, and Hossein Yalame. ABY2.0: improved mixed-protocol secure two-party computation. In *30th USENIX Security Symposium, USENIX Security 2021*, pages 2165–2182. USENIX Association, 2021.

[VBH21] Adithya Vadapalli, Fattaneh Bayatbabolghani, and Ryan Henry. You may also like... privacy: Recommendation systems meet PIR. *Proc. Priv. Enhancing Technol.*, 2021(4):30–53, 2021.

[VHG23] Adithya Vadapalli, Ryan Henry, and Ian Goldberg. Duoram: A bandwidth-efficient distributed ORAM for 2- and 3-party computation. In *32nd USENIX Security Symposium, USENIX Security 2023*, pages 3907–3924. USENIX Association, 2023.

[VSH22] Adithya Vadapalli, Kyle Storrier, and Ryan Henry. Sabre: Sender-anonymous messaging with fast audits. In *43rd IEEE Symposium on Security and Privacy, SP 2022*, pages 1953–1970. IEEE, 2022.

[ZPZP25] Jinhao Zhu, Liana Patel, Matei Zaharia, and Raluca Ada Popa. Compass: Encrypted semantic search with high accuracy. In *19th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2025*, pages 915–938. USENIX Association, 2025.

[ZSF25] Mingxun Zhou, Elaine Shi, and Giulia Fanti. Pacmann: Efficient private approximate nearest neighbor search. In *The Thirteenth International Conference on Learning Representations, ICLR 2025*. OpenReview.net, 2025.

A Protocols

The algorithms below follow the code line by line where it matters, including the points at which rounds are charged. $[\![x]\!]$ is a 2-of-3 replicated sharing over $\mathbb{Z}_{2^b}$, and $\gg_{\text{pair}}$ denotes a call to Algorithm 2.

Algorithm 1 ApproxFactor under sharing (`src/mf/factor.cpp`).

Require: $[\![\mathbf{U}]\!]$ at scale 0, parameters d, ℓ, t, a truncation schedule proven to fit, a normaliser Normalize
Ensure: $\mathbf{B} \in \mathcal{R}^{d \times n}$, public

$[\![\mathbf{U}^\top]\!] \leftarrow$ local relabelling of $[\![\mathbf{U}]\!]$ ▷ 0 rounds
for $c = 1, \ldots, d$ **do**
  $[\![\mathbf{v}]\!] \leftarrow$ shares of a small random start vector
  $[\![\mathbf{v}]\!] \leftarrow \text{Normalize}(\text{SetOrthogonal}([\![\mathbf{v}]\!], \mathbf{B}_{1..c-1}))$
  for $j = 1, \ldots, \ell$ **do**
    $[\![\mathbf{w}]\!] \leftarrow [\![\mathbf{U}]\!][\![\mathbf{v}]\!]$ ▷ 1 round, $3m$ elements
    $[\![\mathbf{v}]\!] \leftarrow [\![\mathbf{U}^\top]\!][\![\mathbf{w}]\!]$ ▷ 1 round, $3n$ elements
    $[\![\mathbf{v}]\!] \leftarrow [\![\mathbf{v}]\!] \gg_{\text{pair}} s$ ▷ public shift s from the schedule, 3 rounds
    $[\![\mathbf{v}]\!] \leftarrow \text{SetOrthogonal}([\![\mathbf{v}]\!], \mathbf{B}_{1..c-1})$ ▷ local products, then 1 or 2 truncations
    $[\![\mathbf{v}]\!] \leftarrow \text{Normalize}([\![\mathbf{v}]\!])$
  $\mathbf{B}_c \leftarrow \text{Open}([\![\mathbf{v}]\!])$ ▷ 1 round, leaked by design
return B ▷ $[\![\mathbf{A}]\!] = [\![\mathbf{U}]\!]\,\mathbf{B}^\top$ is then local

Algorithm 2 TruncatePair, truncation by t bits with helper h (`src/mpc/nonlinear/trunc.cpp`). Parties a and b are the other two.

Require: $[\![x]\!]$, shift t, helper index h
Ensure: $[\![y]\!]$ with $y = x \gg t$ up to one unit

h draws uniform r and sends a and b additive shares of r and of $r \gg t$ ▷ round 1, 4 elements
a and b exchange masked values and both learn $c = x - r$ ▷ round 2, 2 elements
a sets $y_a \leftarrow (c \gg t) + (r \gg t)_a$ and b sets $y_b \leftarrow (r \gg t)_b$
a and b draw a pairwise mask μ that h cannot compute, set y_a += μ and y_b −= μ, and h advances its generator to stay in step
reshare $(y_a, y_b, 0)$ into replicated form ▷ round 3, 2 elements

Algorithm 3 The MSNZB gate and the shared inverse square root (`src/mpc/nonlinear/msnzb.cpp`).

Require: $[\![S]\!]$ with $\text{msnzb}(S) \in [lo, hi]$, a bound the caller must guarantee, and Newton step count N
Ensure: $[\![y]\!]$ encoding $1/\sqrt{S}$ at scale t, where S is itself at scale t

Offline, by a dealer:
draw r uniform on $[0, 2^{L+\kappa})$ with $\kappa = 40$
for $k = lo, \ldots, hi$ **do**
  DCF keys for the threshold $2^k + r$ over a domain of $L + \kappa + 1$ bits

Online:
open $c = S + r$ ▷ round 1, every threshold shares it
each party evaluates every key at c, giving shares of $1[S \geq 2^k]$ for all k
$[\![y]\!] \leftarrow \sum_k \delta_k [\![1[S \geq 2^k]]\!]$, where the public δ_k make $y = 2^{\lfloor (3t - \text{msnzb}(S))/2 \rfloor}$
reshare $[\![y]\!]$ ▷ round 2
for $i = 1, \ldots, N$ **do**
  $[\![y]\!] \leftarrow [\![y]\!]\,(3 - [\![S]\!][\![y]\!]^2)/2$ ▷ 3 products and 3 truncations, 12 rounds
return $[\![y]\!]$ ▷ $N = 4$: 50 rounds, worst relative error 1.41×10^{-6}

Algorithm 4 The harvest and the weight check (`src/pir/harvest.cpp`).

Require: for each PIR query, the expansions $e_0 = \text{EvalFull}(k_0)$ at P_0 and $e_1 = \text{EvalFull}(k_1)$ at P_1

Per query, the weight check:

P_0 computes $s_0 = \sum_j e_0[j]$ and P_1 computes $s_1 = \sum_j e_1[j]$ ▷ local

exchange s_0, s_1 and reject the query unless $s_0 - s_1 = 1$ ▷ 1 round, 2 elements

h_0 += e_0 at P_0 and h_1 += e_1 at P_1 ▷ local

Once per training round, the conversion:

for each item j **do**

    P_0 and P_1 draw a pairwise mask μ_j, and P_2 advances its generator

    $x_0 \leftarrow h_0[j] + \mu_j, \quad x_1 \leftarrow -h_1[j] - \mu_j, \quad x_2 \leftarrow 0$

reshare (x_0, x_1, x_2) into replicated form, so that $x_0 + x_1 + x_2 = h_0 - h_1$ ▷ 1 round, $3n$ elements

B Reproducing every figure and number

Every figure in this report is drawn by `mingw32-make figures` from raw rows committed under `bench/results/`, and none is edited by hand. Each row carries the commit it was produced at, and the figure script uses the newest commit's rows in each file. One command rebuilds everything from a clean checkout, on MSYS2's g++ and Python 3.13:

```
mingw32-make reproduce
```

It fetches MovieLens-100K, builds, runs all eighteen test binaries, trains the eighteen private models, runs the studies and benchmarks, and draws the figures and this report. It takes on the order of a quarter of an hour on the development laptop, and `mingw32-make reproduce-bg` followed by `mingw32-make watch` runs it in the background with a progress bar. Table 11 gives the producer of each figure's data, for a reader who wants to rerun one piece.

Table 11: Where each figure's data comes from.

Figure	Data file in `bench/results/`	Written by
2, key size	none, a closed form	asserted by `tests/test_dpf_serialize.cpp`
3, quality against ℓ	`oracle_ell_sweep.jsonl`	checked by `model/ell_sweep.py`
4, each half isolated	`bench_compose.jsonl`	`mingw32-make bench`
5, the read against baselines	`bench_baseline.jsonl`	`mingw32-make bench`
6, scaling	`bench_sweep.jsonl`	`mingw32-make bench`
7, where rounds go	`bench_stages.jsonl`	`mingw32-make bench`
8, wire distinguisher	`distinguisher_result.jsonl`	`mingw32-make distinguisher`
9, server timing	`bench_timing.jsonl`	`mingw32-make bench`
10, reconstruction attack	`leakage_attack.jsonl`	`model/attack.py`
11, decoys	`leakage_attack.jsonl`	`model/attack.py`
12, differential privacy	`dp_study.jsonl`	`model/dp_study.py`
13, malicious client	`poison_study.jsonl`, `bench_dos.jsonl`	`model/poison_study.py`, `mingw32-make bench`

Three further sources back numbers that no figure shows. Private-training quality (Table 5) is in `train_quality.jsonl`, written by `model/score_private.py` from the eighteen trained models. The ring-width study is in `train.jsonl`, written by `train --headroom-only`. And every number quoted from a test, such as the 227-byte key, the one-round weight check, the truncation error bound and the inverse square root's error, is printed by `mingw32-make test`, which must pass for any of the above to run.

Two caveats on rerunning. The oracle's ℓ sweep is the one data file written by a one-off run rather than by a script in the pipeline. The pipeline therefore recomputes it and checks every committed row instead of appending new ones, and all six rows reproduce exactly. And rerunning a benchmark appends rows at a new commit, which the figures then use, so absolute times in a rerun will differ from the ones quoted here by the thermal variation Section 6.1 describes. Rounds, bytes and quality will not.